Tema 7

Uso de bases de datos objeto relacionales

Base de datos

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Tipos de bases de datos orientadas a objetos 5
2.1 Instalación de PostgreSQL 7
2.2. Acceso a PostgreSQL 19
3. Características de las bases de datos objeto-relacionales 23
4. Tipos de datos objeto 27
4.1 Tipos de datos nativos en PostgreSQL 27
4.2. Tipos de datos personalizados 28
5. Definición de tipos de objeto 30
6. Herencia 34
6.1 Herencia de tablas 34
6.2. Restricciones a la herencia en PostgreSQL 36
6.3. Técnicas de herencia múltiple 38
7. Identificadores; referencias. Navegabilidad 41
8. Tipos de datos colección 44
9. Declaración e inicialización de objetos 46
10. Inserción de objetos 48
11. Uso de la sentencia SELECT 50
12. Modificación y borrado de objetos 52
13. Resumen 55
Glosario 56
Bibliografía 57

Objetivos de la unidad

Las bases de datos objeto-relacionales tienen como objetivo principal proporcionar una mayor flexibilidad en el modelado de datos y en la manipulación de objetos complejos, asimilando su estructura a la utilizada en los lenguajes de programación que se basan en el paradigma de la programación orientada a objetos.

Algunos de los objetivos específicos de las bases de datos objeto-relacionales son:

- **Integrar la orientación a objetos con las bases de datos relacionales:** Buscan combinar las ventajas de ambos paradigmas, lo que permite trabajar con objetos complejos en un entorno de bases de datos relacional.
- **Soportar tipos de datos complejos:** Permiten definir nuevos tipos de datos complejos como, por ejemplo, matrices, imágenes, archivos de audio y vídeo, etc. Estos tipos de datos no están disponibles en las bases de datos relacionales convencionales.
- **Extender el modelo relacional:** Amplían el modelo relacional tradicional, permitiendo definir nuevos tipos de relaciones entre tablas, como herencia, agregación y composición.
- **Aumentar la eficiencia en el acceso a los datos:** Persiguen mejorar la eficiencia en el acceso a los datos, proporcionando mecanismos para optimizar las consultas que involucran objetos complejos.
- **Proporcionar una mayor flexibilidad en el modelado de datos:** Permiten un mayor grado de flexibilidad en el modelado de datos, lo que las hace adecuadas para aplicaciones donde los requisitos de datos son muy cambiantes o impredecibles.

1. Introducción

Una base de datos objeto-relacional es un tipo de base de datos que combina la capacidad de las bases de datos relacionales tradicionales para almacenar datos en forma de tablas con la capacidad de las bases de datos orientadas a objetos para trabajar con datos complejos y objetos.

En las bases de datos objeto-relacionales, los datos se almacenan en tablas que contienen campos y relaciones entre tablas. Sin embargo, además de los tipos de datos simples (como números y cadenas de texto), las bases de datos objeto-relacionales permiten la definición de tipos de datos complejos, como objetos, colecciones, matrices, imágenes y otros tipos de datos complejos.

Esto permite a los desarrolladores modelar los datos de manera más precisa y natural, lo que a su vez mejora la eficiencia en la manipulación de los datos. Las bases de datos objeto-relacionales también permiten la definición de nuevas relaciones entre tablas, como herencia, composición y agregación.

Otra ventaja de las bases de datos objeto-relacionales es que proporcionan mecanismos para definir y ejecutar funciones y procedimientos almacenados, lo que puede mejorar la eficiencia en el acceso a los datos.

Las bases de datos objeto-relacionales se utilizan en aplicaciones donde los datos son complejos y cambiantes, como en sistemas de gestión de contenidos web, aplicaciones de comercio electrónico y sistemas de información geográfica. Sin embargo, debido a su complejidad, las bases de datos objeto-relacionales requieren un mayor nivel de experiencia en su diseño y administración.

2. Tipos de bases de datos orientadas a objetos

Existen diferentes tipos de bases de datos relacionales orientadas a objetos (**ORDBMS**, por sus siglas en inglés), cada uno con sus propias características y ventajas. Algunos de los tipos de bases de datos relacionales orientadas a objetos más comunes son:

- **Bases de datos híbridas:** Estas bases de datos combinan los conceptos de las bases de datos relacionales y las orientadas a objetos, permitiendo la creación de tipos de datos complejos, como objetos, matrices y colecciones, pero manteniendo la estructura de tablas y relaciones de las bases de datos relacionales.
- **Bases de datos extensibles:** Estas bases de datos permiten la definición de nuevos tipos de datos y operaciones, lo que las hace muy flexibles y adaptables a diferentes aplicaciones y requisitos de datos.
- **Bases de datos objeto-relacionales puras:** Estas bases de datos proporcionan una integración completa entre los modelos relacionales y orientados a objetos, lo que permite la definición de tipos de datos complejos y relaciones entre tablas más precisas y naturales.
- **Bases de datos orientadas a objetos:** Estas bases de datos almacenan y manipulan objetos directamente, sin utilizar tablas y relaciones. Esto las hace ideales para aplicaciones que trabajan con objetos complejos, como sistemas de información geográfica y aplicaciones multimedia.

Cada tipo de base de datos tiene sus propias ventajas y desventajas, y la elección dependerá de los requisitos específicos de cada aplicación.

Importante

Siempre deberemos tener en cuenta que las bases de datos orientadas a objetos pueden requerir un mayor nivel de conocimientos y habilidades en su diseño y administración, en comparación con las bases de datos relacionales convencionales.

Ejemplos prácticos por tipo de base de datos

1. **Bases de datos híbridas (ORDBMS)**

Caso: Gestión de catálogo de productos con atributos variables

Una tienda online quiere manejar millones de productos, cada uno con características (tallas, colores, dimensiones, etiquetado) que varían según la categoría. Con un ORDBMS híbrido (p. ej. PostgreSQL) pueden:

- Modelar la información común (nombre, precio, stock) en tablas relacionales.
- Almacenar atributos específicos (dimensiones de muebles, composición textil, especificaciones técnicas) como tipos compuestos (JSON, arrays, hstore) o UDTs. Así mantienen integridad relacional y a la vez flexibilidad para esquemas "semi-estructurados" sin necesidad de esquemas EAV complejos.

2. **Bases de datos extensibles**

Caso: Plataforma de investigación genética

Un laboratorio de genómica necesita almacenar no solo secuencias de ADN, sino también nuevas formas de representar variantes genéticas, anotaciones de variantes y algoritmos de alineamiento. Con un sistema extensible (p. ej. Informix con extensiones UDR):

- Definen tipos de datos «SecuenciaADN», «VariaciónGenética» o «Alineamiento».
- Implementan funciones en C/Python para manipularlos (búsquedas de patrones, cálculos de similitud). De ese modo cada nuevo descubrimiento en biología molecular se incorpora al modelo sin rediseñar por completo la base de datos.

3. **Bases de datos objeto-relacionales puras**

Caso: Sistema CAD/CAM para diseño mecánico

Una empresa de ingeniería debe almacenar piezas mecánicas con geometría compleja (sólidos, superficies, ensamblajes) y sus relaciones (herencia de tipo, versiones, restricciones de montaje). Una ORDBMS pura (p. ej. Informix Object-Relational) permite:

- Definir directamente tipos objeto (e.g. class Pieza { Surface[]; Material; }).
- Aprovechar herencia de clases para "PiezaBase" → "Engranaje", "Eje".

- Ejecutar consultas espaciales y reconstruir ensamblajes íntegros. Esto resulta más natural y más fácil de mantener que hacerlo con tablas puramente relacionales.

4. **Bases de datos orientadas a objetos (OODBMS puras)**

Caso: Motor de juego 3D

Un estudio crea un videojuego con mundos tridimensionales, donde cada "entidad" (jugador, NPC, objeto) es un objeto con múltiples métodos y estados internos (move(), render(), colisiones). Un sistema OODBMS puro (p. ej. ObjectStore, db4o) ofrece:

- Almacenamiento y recuperación directa de instancias en memoria, sin mapear manualmente a filas/columnas.
- Persistencia transparente de jerarquías de clases, polimorfismo y punteros de objeto.
- Máxima eficiencia en acceso a grafos complejos de objetos enlazados.

2.1 Instalación de PostgreSQL

Para poder seguir los ejemplos y practicar es conveniente instalar algún **SGBD** que además de soportar un sistema de base de datos relacional, soporte el diseño orientado a objetos. Aunque hay varios SGBD que podríamos utilizar, nos decantaremos por **PostgreSQL**.

PostgreSQL es una base de datos objeto-relacional pura que:

- Proporciona una integración completa entre los modelos relacionales y orientados a objetos.
- Admite tipos de datos complejos y relaciones entre tablas más precisas y naturales, lo que la hace ideal para aplicaciones que manejan objetos complejos.
- Proporciona características avanzadas de bases de datos relacionales, como integridad referencial, transacciones y soporte para lenguajes de programación como SQL y PL/pgSQL.

A continuación, vamos a ver como instalar PostgreSQL en nuestro equipo. En primer lugar, deberemos descargar la versión que necesitemos en función de

la arquitectura de nuestro equipo y del sistema operativo en el que vayamos usarla.

Para ello, visitaremos el siguiente enlace y descargaremos la versión deseada:

https://www.enterprisedb.com/downloads/postgres-postgresql-downloads

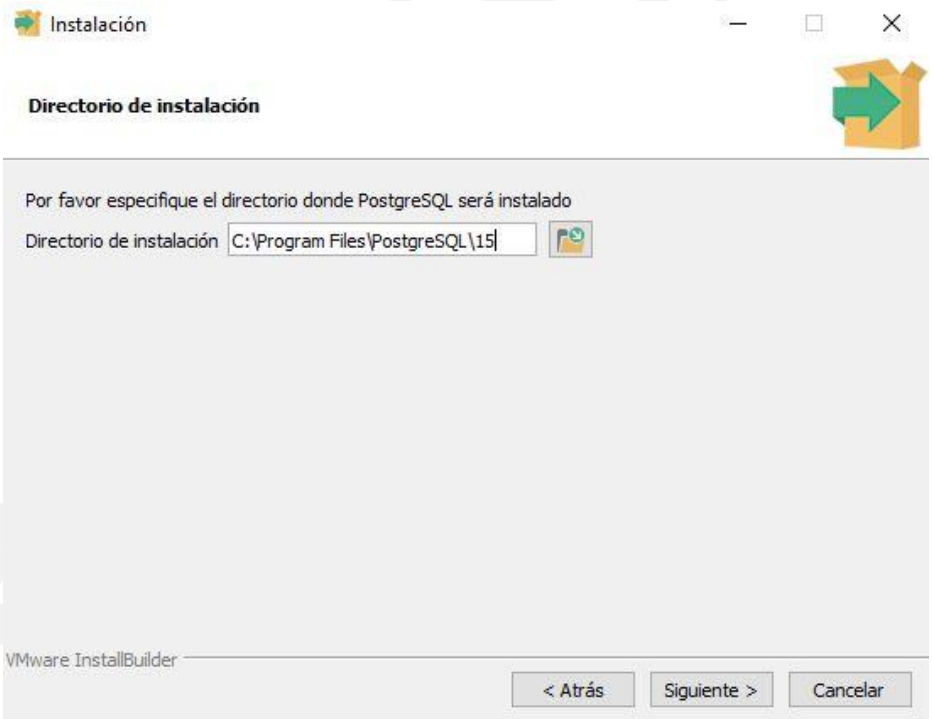
Download PostgreSQL

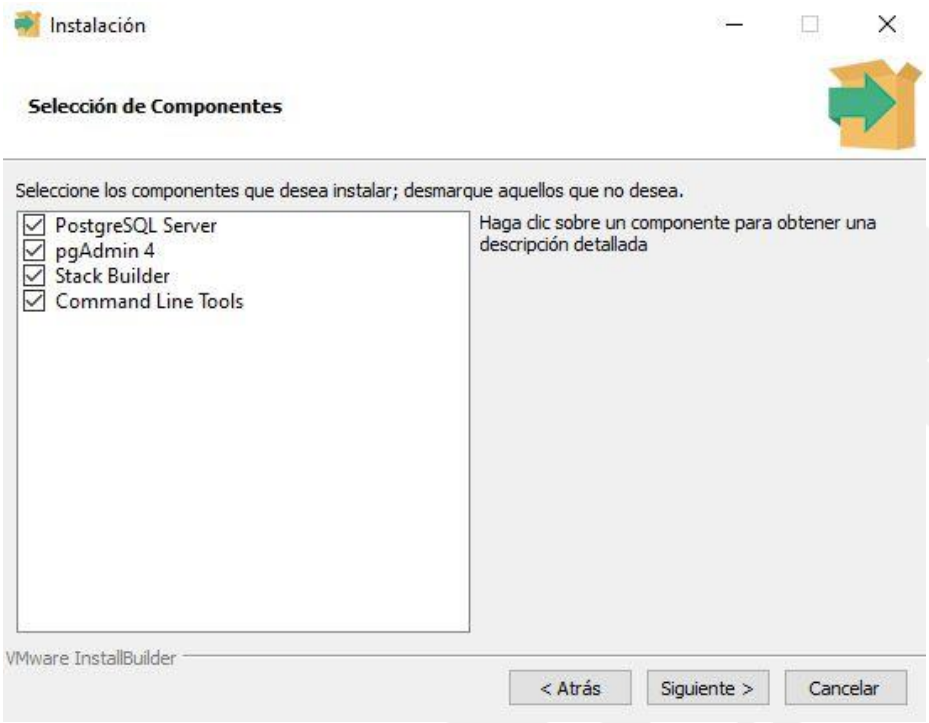
Open source PostgreSQL packages and installers from EDB

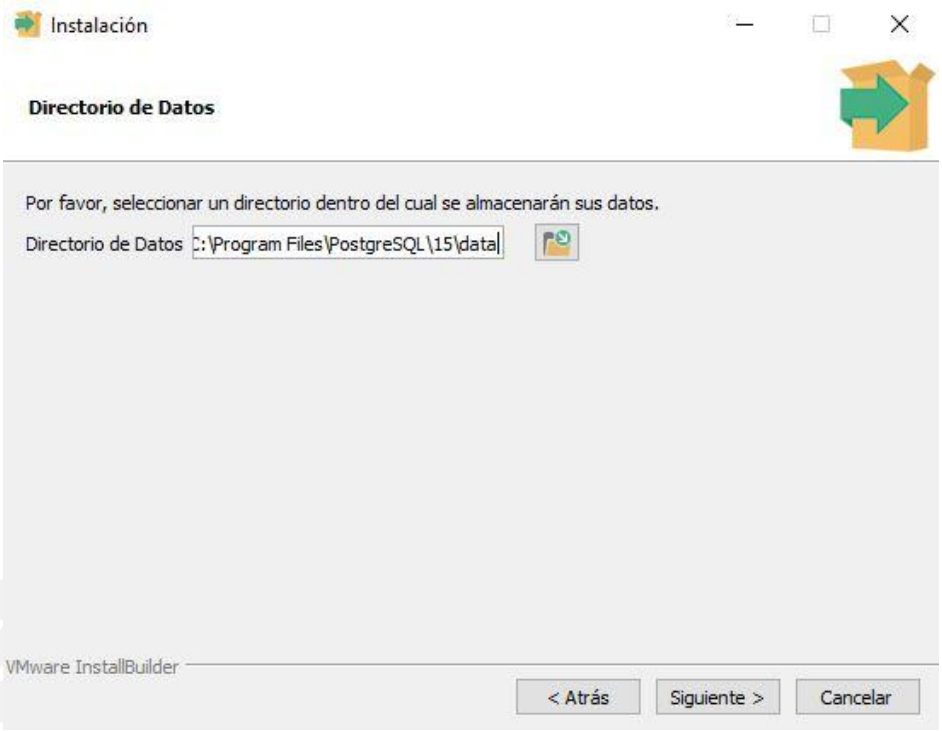
PostgreSQL Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
15.2	postgresql.org	postgresql.org			Not supported
14.7	postgresql.org	postgresql.org			Not supported
13.10	postgresql.org	postgresql.org			Not supported
12.14	postgresql.org	postgresql.org			Not supported
11.19	postgresql.org	postgresql.org			Not supported
10.23*					
9.6.24*					
9.5.25*					
9.4.26*					

En este manual, explicaremos la instalación para un Sistema Operativo **Windows x86-64**. Una vez descargado el archivo de instalación, seguiremos los pasos con el que nos guiará el asistente de instalación:

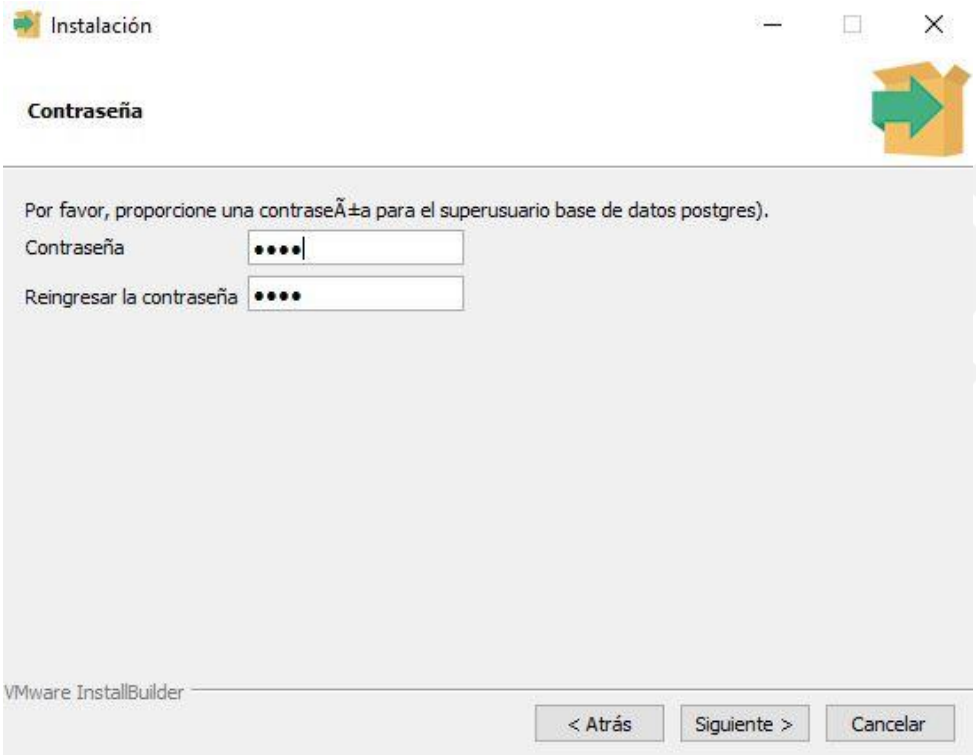
Instalación
PACKAGED BY
EDB
Instalar - PostgreSQL
Bienvenido a la instalación de PostgreSQL.
PostgreSQL
< Atrás
Siguiente >
Cancelar


Instalación
Directorio de instalación
Por favor especifique el directorio donde PostgreSQL será instalado
Directorio de instalación
C:\Program Files\PostgreSQL\15
VMware InstallBuilder
< Atrás
Siguiente >
Cancelar

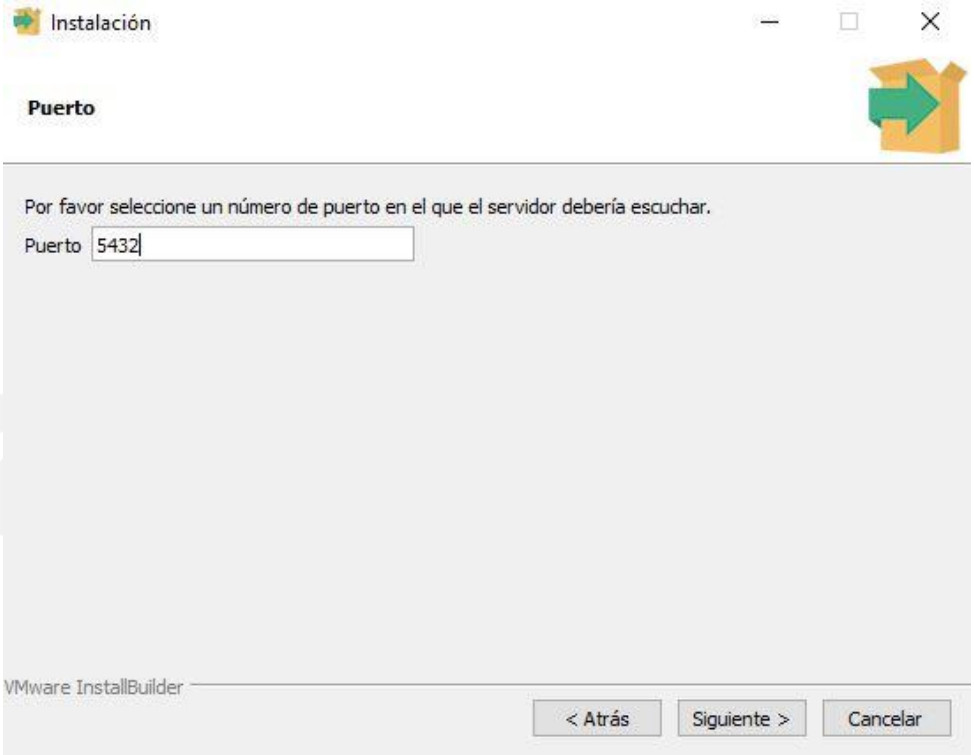


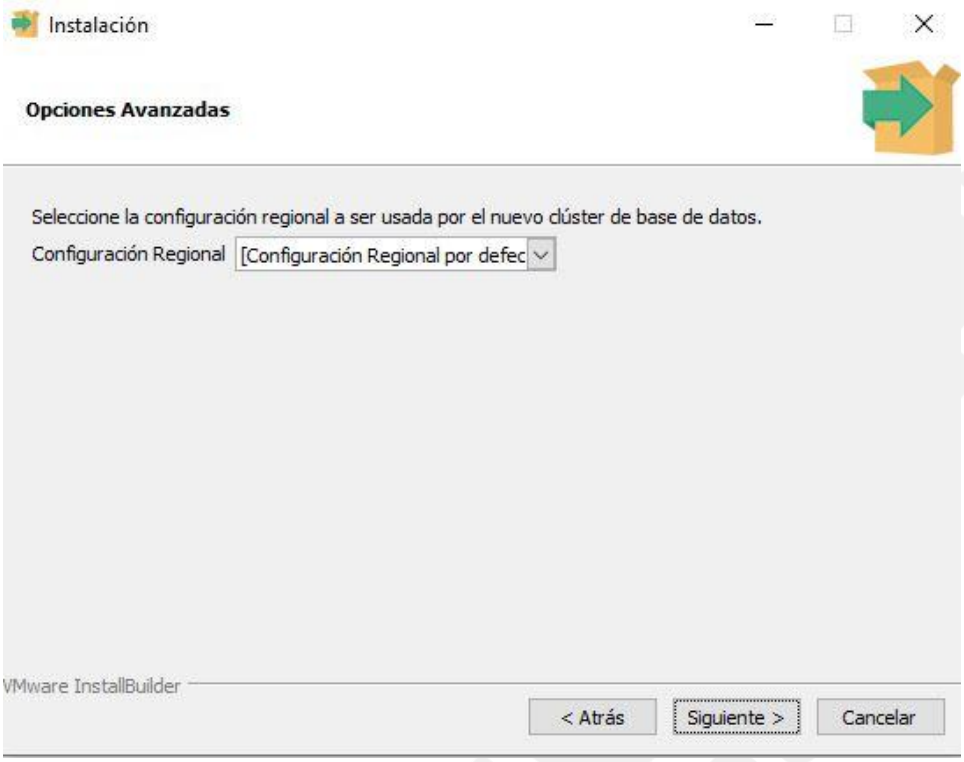


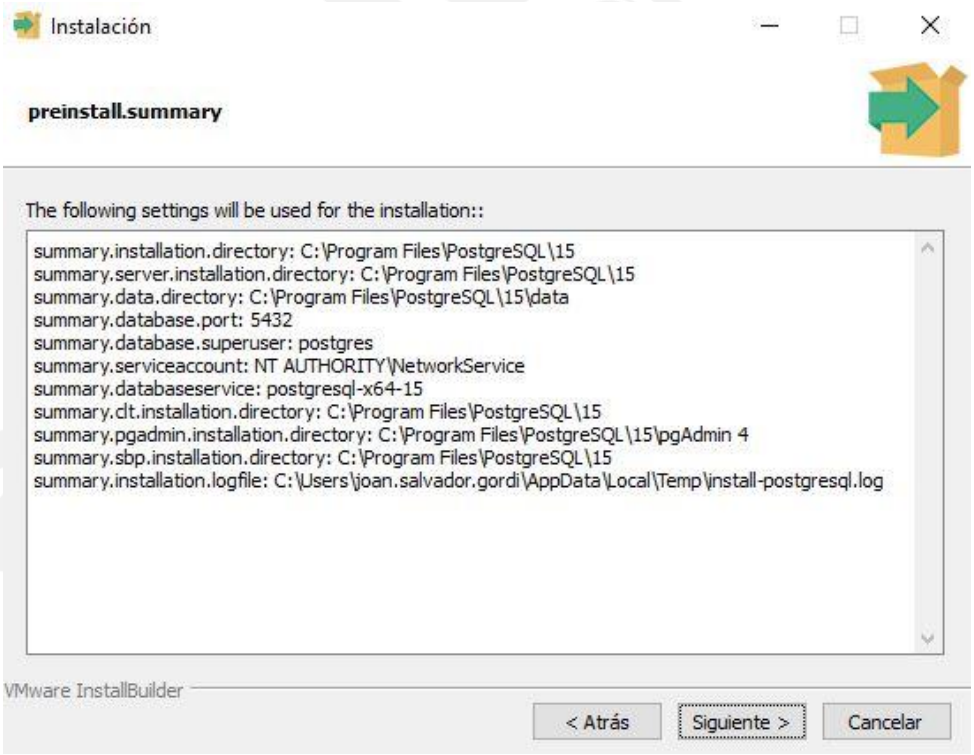
Es importante indicar una contraseña que podamos recordar cuando necesitemos acceder a la base de datos:

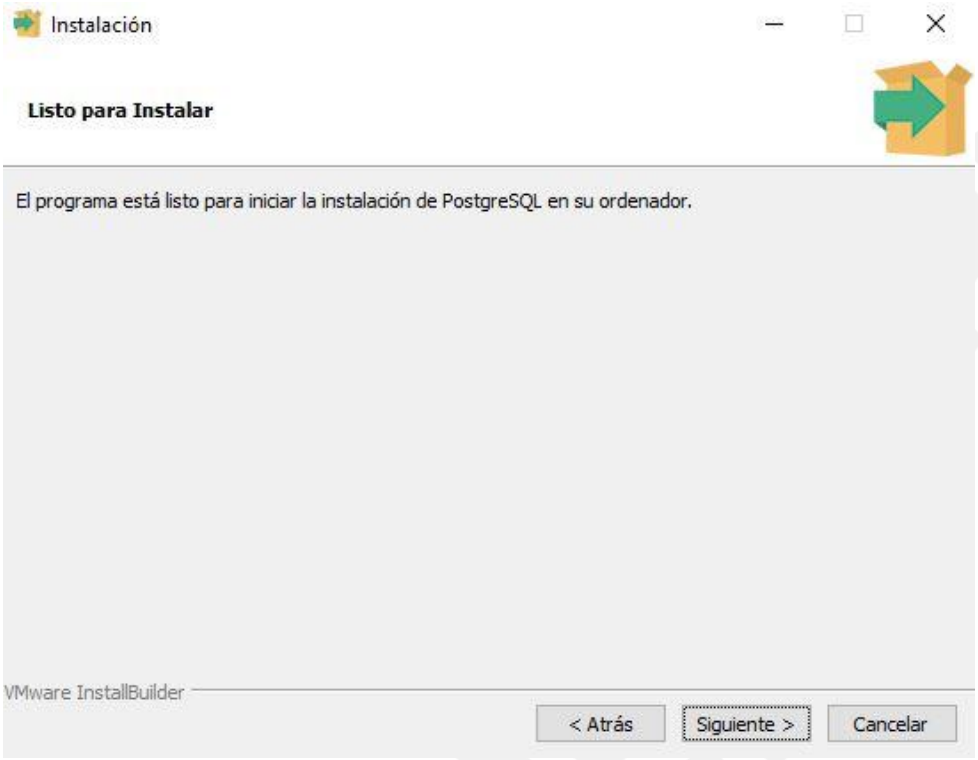


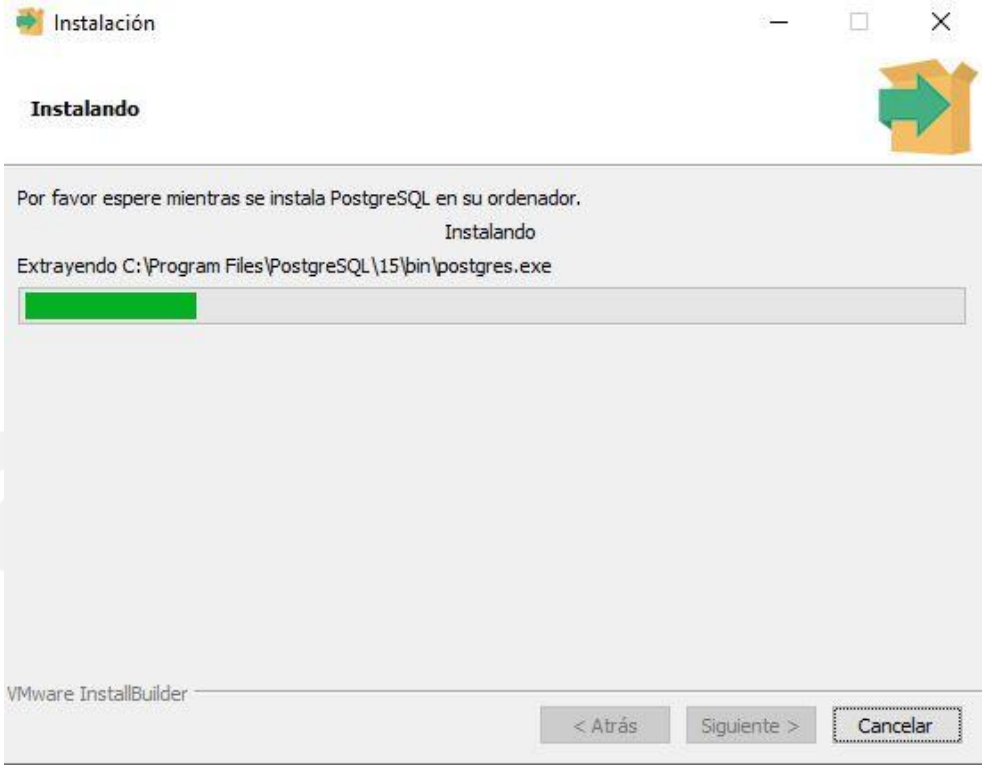
El puerto por defecto de **PostgreSQL** es el **5432**:




Instalación
Opciones Avanzadas
Seleccione la configuración regional a ser usada por el nuevo clúster de base de datos.
Configuración Regional
[Configuración Regional por defec
VMware InstallBuilder
< Atrás
Siguiente >
Cancelar


Instalación
preinstall.summary
The following settings will be used for the installation::
summary.installation.directory: C:\Program Files\PostgreSQL\15
summary.server.installation.directory: C:\Program Files\PostgreSQL\15
summary.data.directory: C:\Program Files\PostgreSQL\15\data
summary.database.port: 5432
summary.database.superuser: postgres
summary.serviceaccount: NT AUTHORITY\NetworkService
summary.databaseservice: postgresql-x64-15
summary.clt.installation.directory: C:\Program Files\PostgreSQL\15
summary.pgadmin.installation.directory: C:\Program Files\PostgreSQL\15\pgAdmin 4
summary.sbp.installation.directory: C:\Program Files\PostgreSQL\15
summary.installation.logfile: C:\Users\joan.salvador.gordi\AppData\Local\Temp\install-postgresql.log
VMware InstallBuilder
< Atrás
Siguiente >
Cancelar

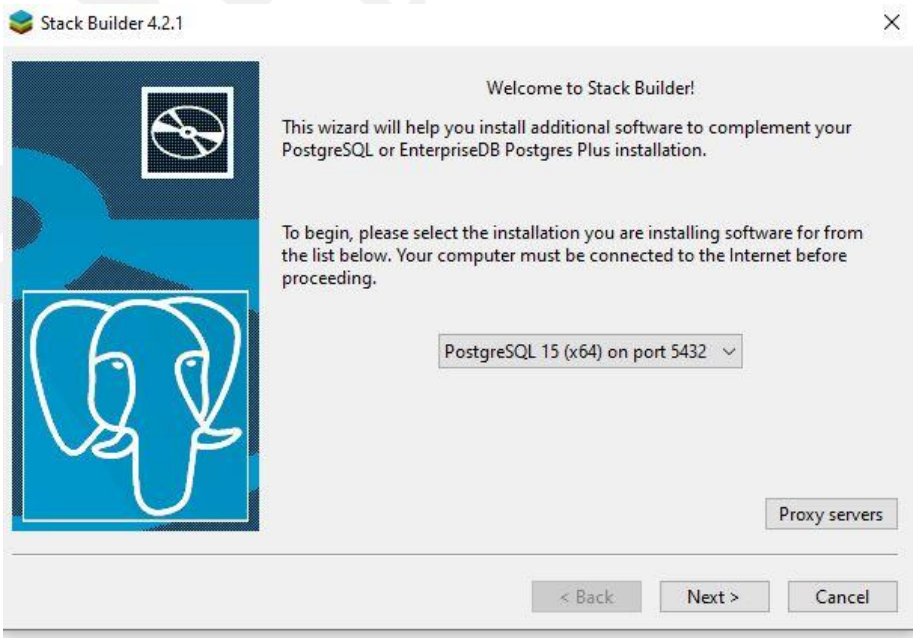

Instalación
Listo para Instalar
El programa está listo para iniciar la instalación de PostgreSQL en su ordenador.
VMware InstallBuilder
< Atrás
Siguiente >
Cancelar

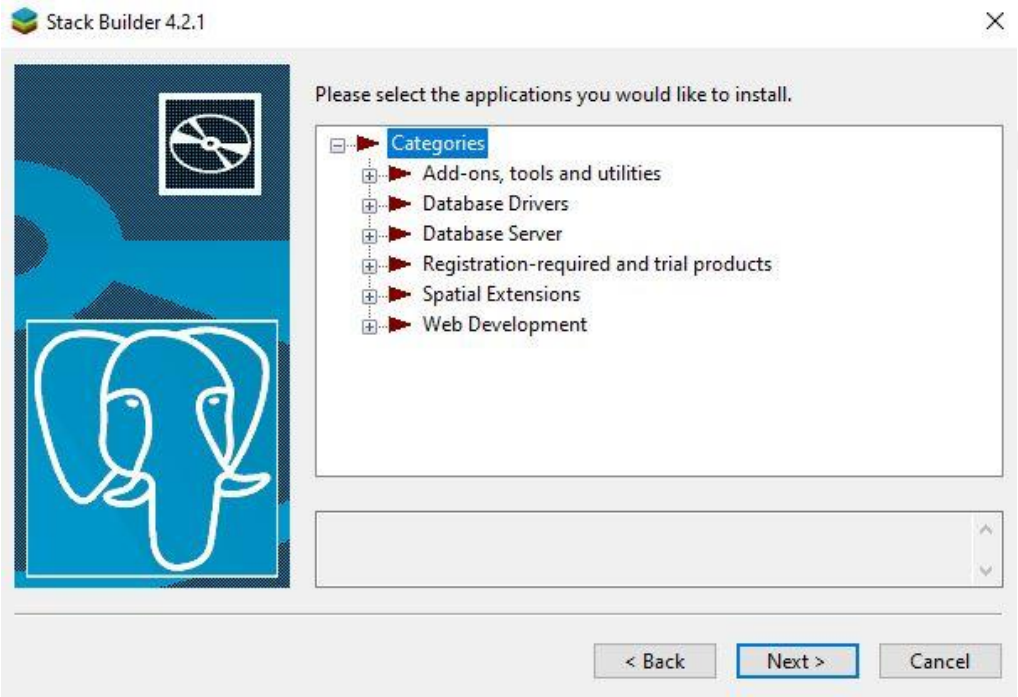

Instalación
Instalando
Por favor espere mientras se instala PostgreSQL en su ordenador.
Instalando
Extrayendo C:\Program Files\PostgreSQL\15\bin\postgres.exe
VMware InstallBuilder
< Atrás
Siguiente >
Cancelar

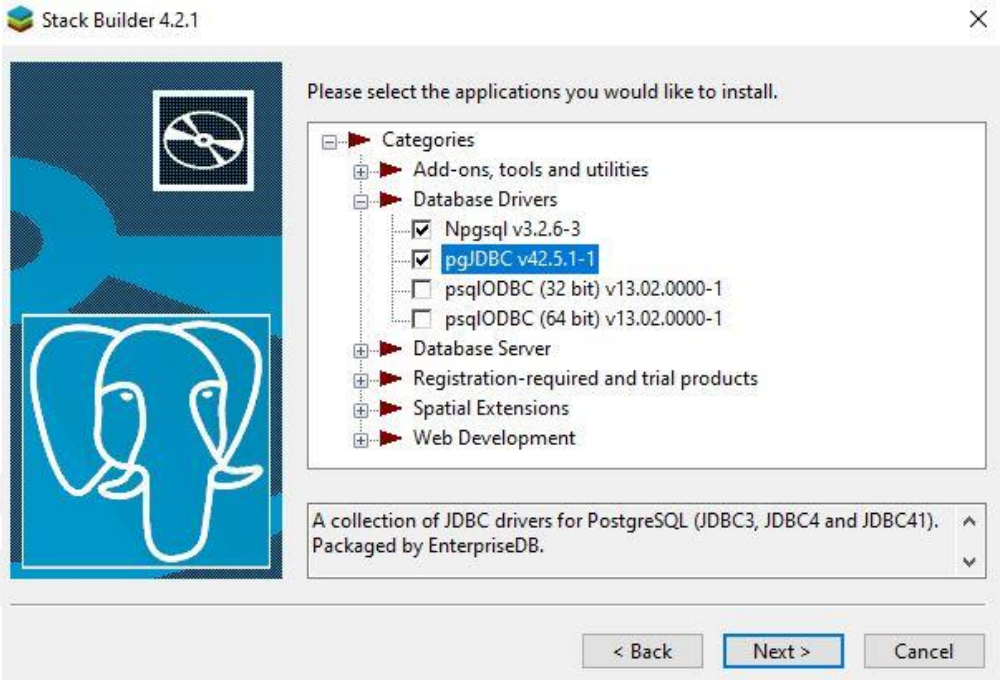
Una vez finalizada la instalación, es recomendable lanzar **Stack Builder**, que nos ayudará a completar la instalación de PostgreSQL con aplicaciones adicionales.

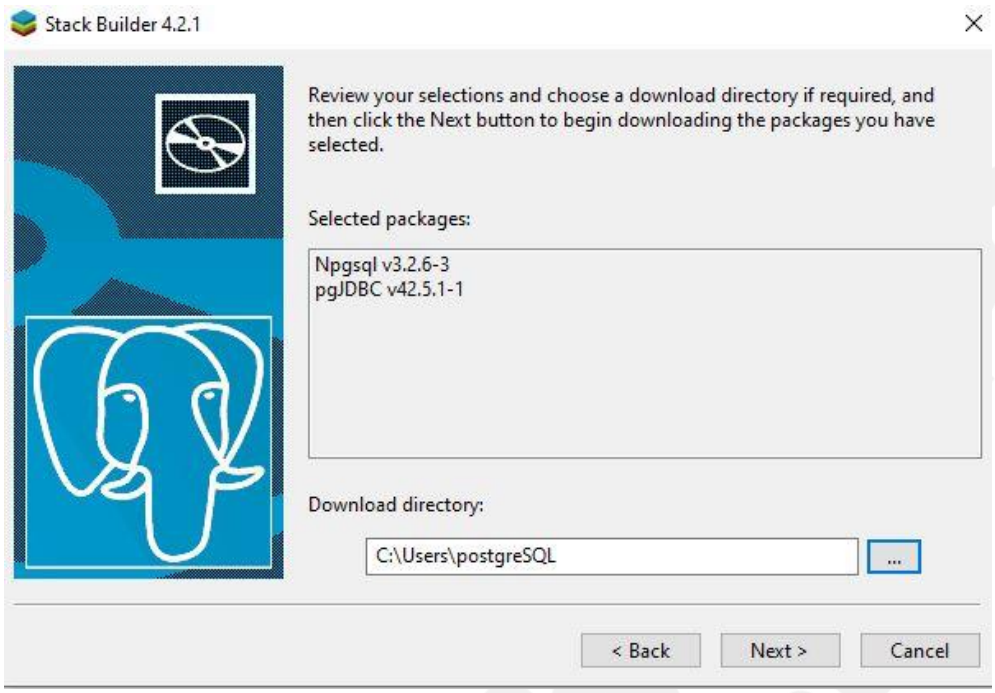
En nuestro caso, y simplemente a muestra de ejemplo, instalaremos los drivers que nos permitirán acceder a bases de datos PostgreSQL desde aplicaciones .NET y JAVA:

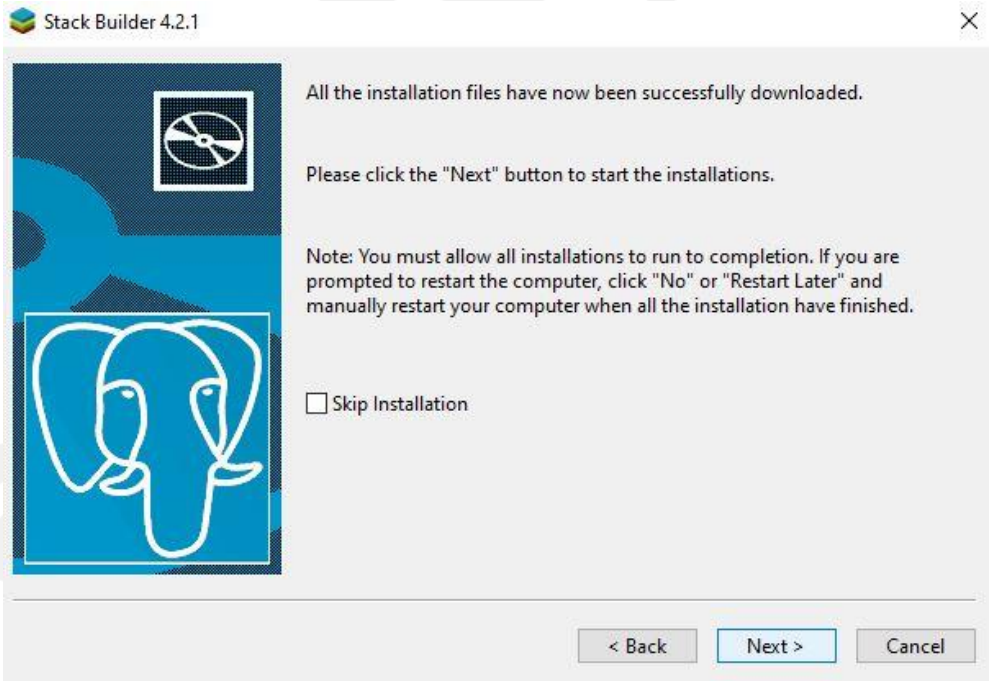
- Npgsql para .NET
- pgJDBC para JAVA

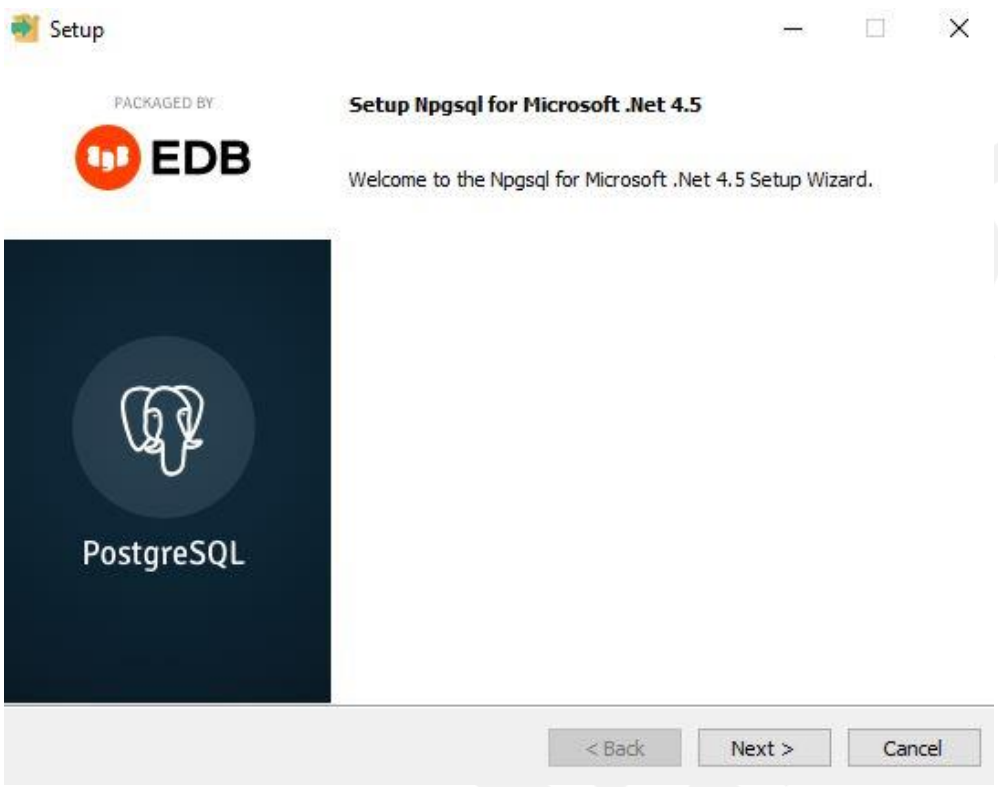


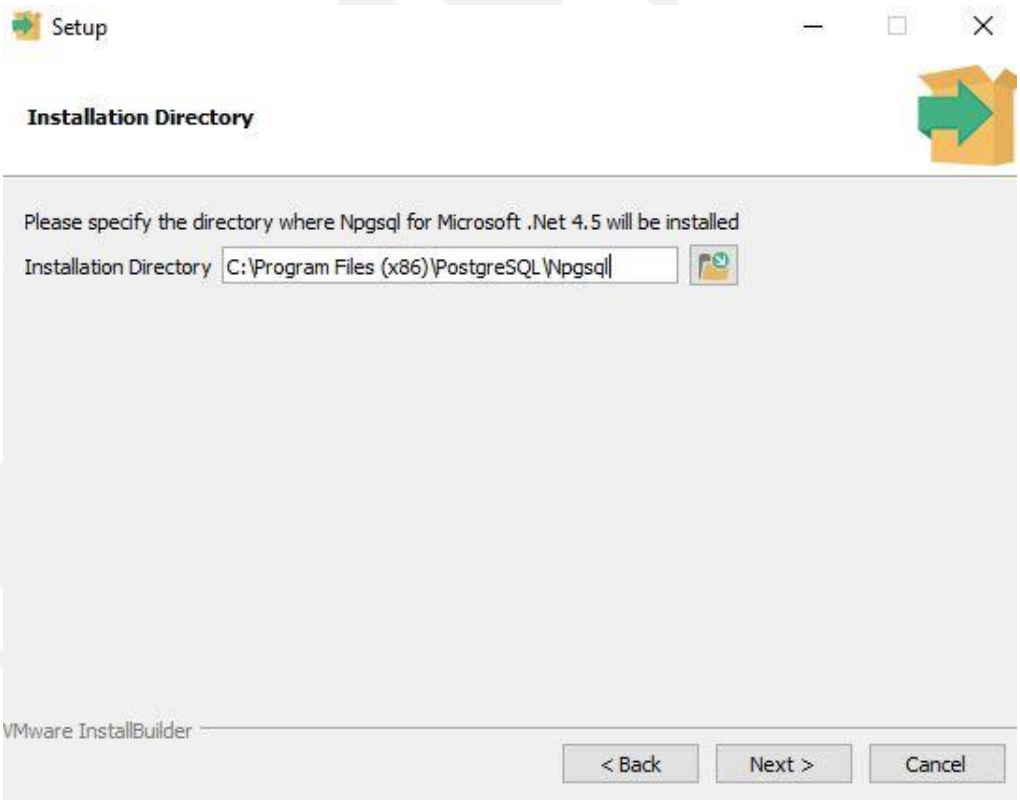

Stack Builder 4.2.1
Please select the applications you would like to install.
Categories
Add-ons, tools and utilities
Database Drivers
Database Server
Registration-required and trial products
Spatial Extensions
Web Development
< Back
Next >
Cancel

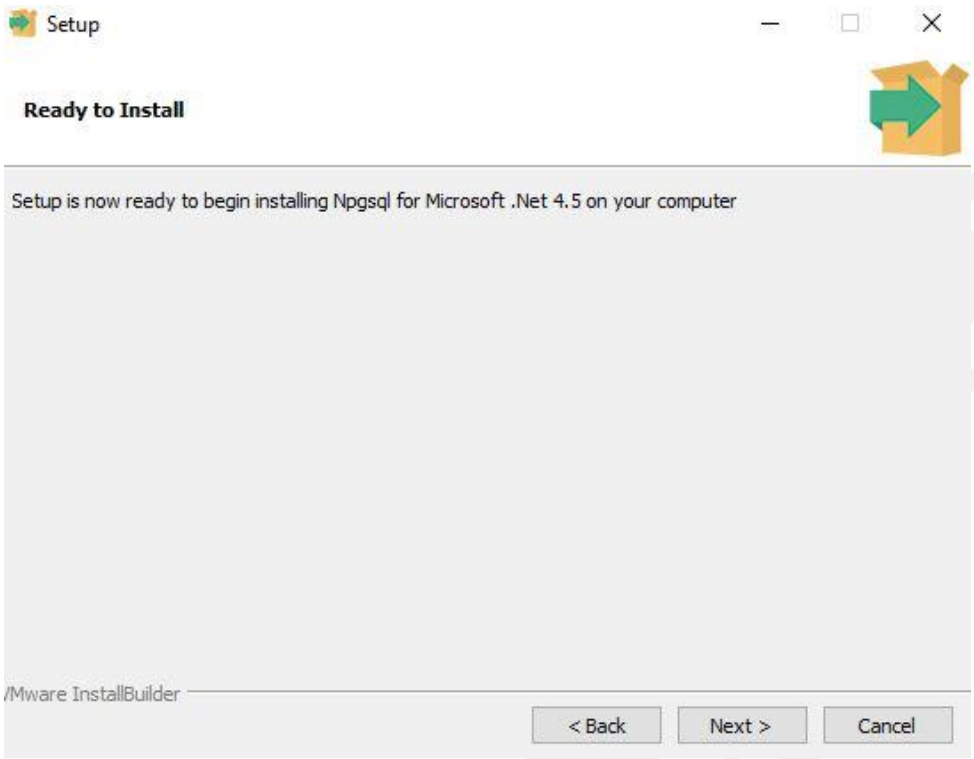

Stack Builder 4.2.1
Please select the applications you would like to install.
Categories
Add-ons, tools and utilities
Database Drivers
Npgsql v3.2.6-3
pgJDBC v42.5.1-1
psqlODBC (32 bit) v13.02.0000-1
psqlODBC (64 bit) v13.02.0000-1
Database Server
Registration-required and trial products
Spatial Extensions
Web Development
A collection of JDBC drivers for PostgreSQL (JDBC3, JDBC4 and JDBC41).
Packaged by EnterpriseDB.
< Back
Next >
Cancel

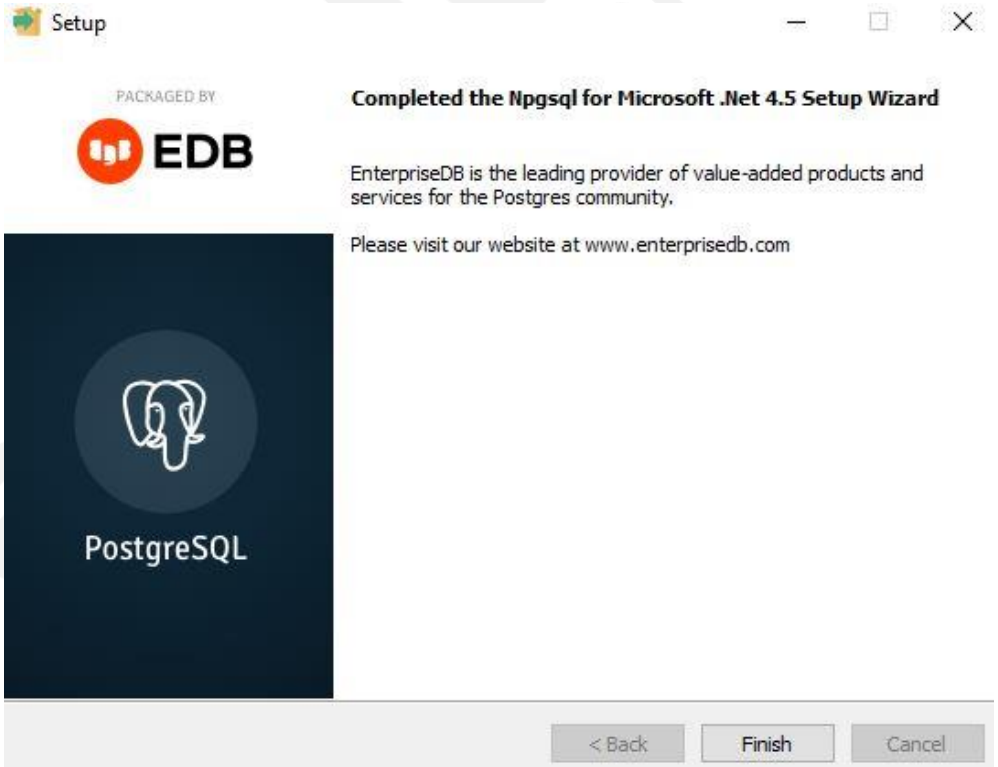

Stack Builder 4.2.1
Review your selections and choose a download directory if required, and then click the Next button to begin downloading the packages you have selected.
Selected packages:
Npgsql v3.2.6-3
pgJDBC v42.5.1-1
Download directory:
C:\Users\postgreSQL
...
< Back
Next >
Cancel

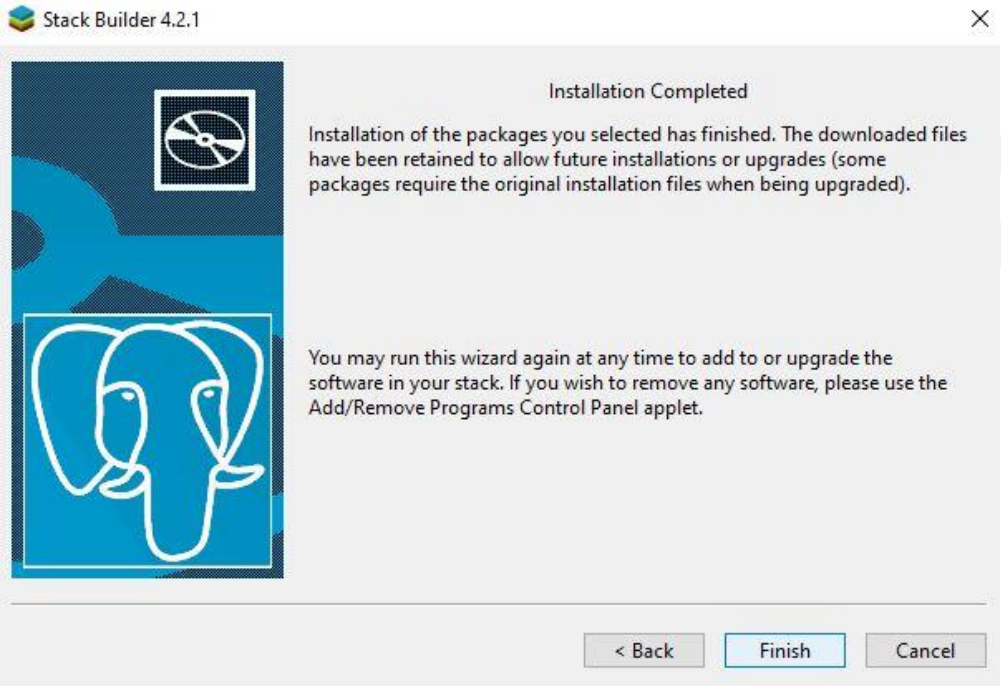

Stack Builder 4.2.1
All the installation files have now been successfully downloaded.
Please click the "Next" button to start the installations.
Note: You must allow all installations to run to completion. If you are prompted to restart the computer, click "No" or "Restart Later" and manually restart your computer when all the installation have finished.
Skip Installation
< Back
Next >
Cancel


Setup
PACKAGED BY
EDB
Setup Npgsql for Microsoft .Net 4.5
Welcome to the Npgsql for Microsoft .Net 4.5 Setup Wizard.
PostgreSQL
< Back
Next >
Cancel


Setup
Installation Directory
Please specify the directory where Npgsql for Microsoft .Net 4.5 will be installed
Installation Directory
C:\Program Files (x86)\PostgreSQL\Npgsql
VMware InstallBuilder
< Back
Next >
Cancel

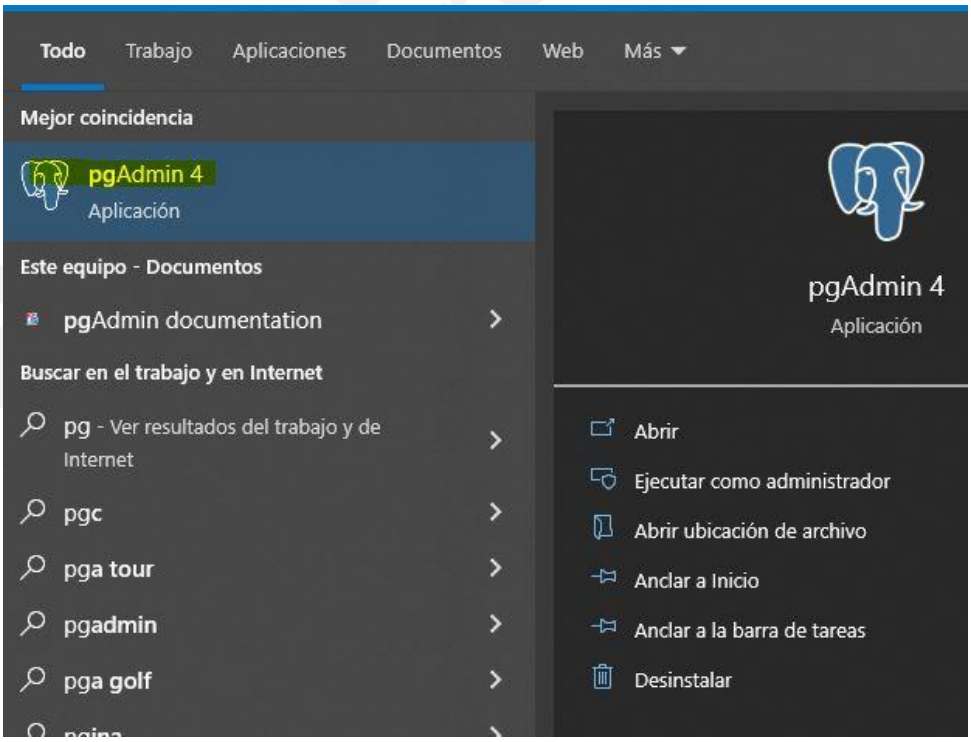

Setup
Ready to Install
Setup is now ready to begin installing Npgsql for Microsoft .Net 4.5 on your computer
/Mware InstallBuilder
< Back
Next >
Cancel

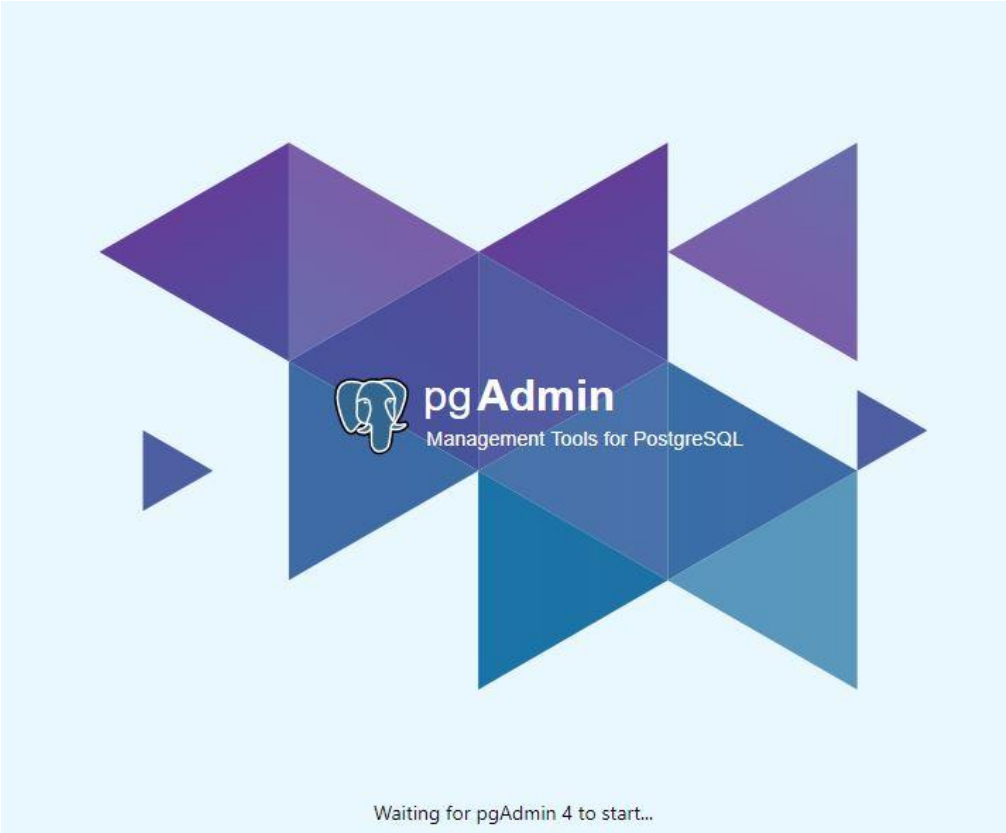

Setup
PACKAGED BY
EDB
Completed the Npgsql for Microsoft .Net 4.5 Setup Wizard
EnterpriseDB is the leading provider of value-added products and services for the Postgres community.
Please visit our website at www.enterprisedb.com
PostgreSQL
< Back
Finish
Cancel



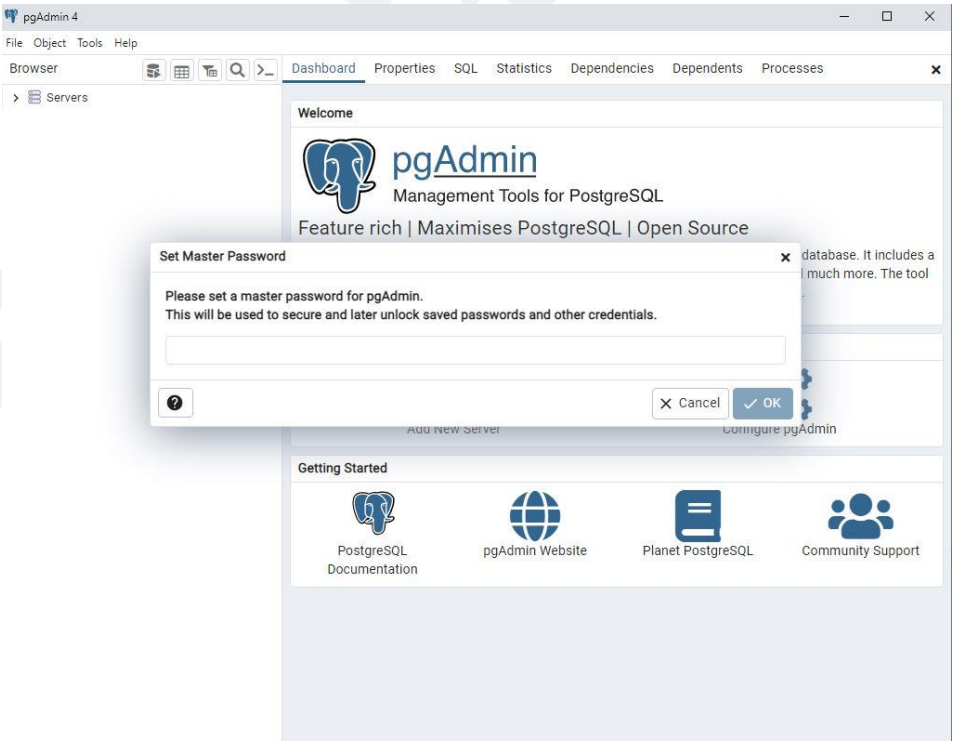
2.2. Acceso a PostgreSQL

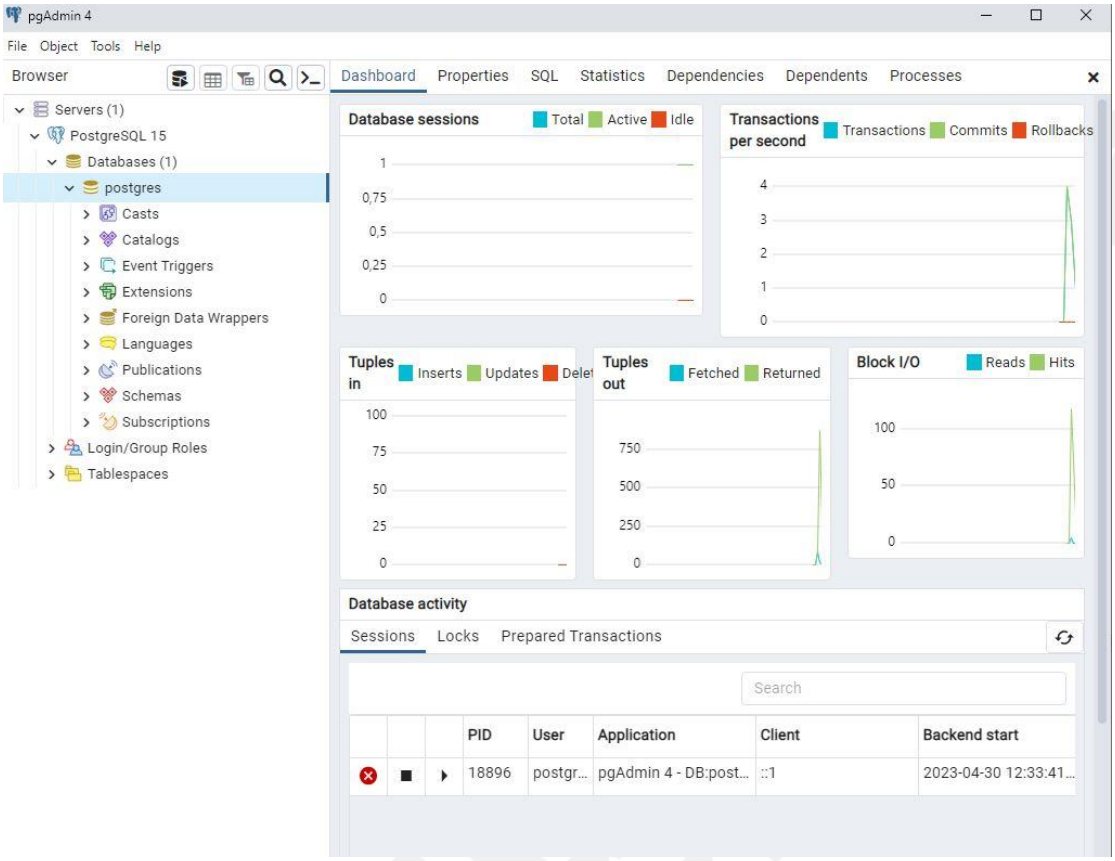
Una vez finalizada la instalación, y cada vez que queramos acceder a nuestro SGBD, tendremos que hacerlo mediante **pgAdmin4**, que es una de las aplicaciones que se instalaron durante el proceso anterior:





Introduciremos el password, y accederemos al administrador de servidores (en cada servidor habrá que volver a introducir el password) y sus bases de datos:





Si analizamos la estructura que nos propone el SGBD, veremos que se estructura en **Servidores**, que a su vez se estructuran en **Logins/Grup Roles**, **Tablespaces**, y **Bases de datos**, y estas a su vez en **tablas**, **funciones**, ..., y **tipos**, que es lo que nos permitirá trabajar con el paradigma orientado a objetos:

Browser
Servers (1)
PostgreSQL 15
Databases (1)
postgres
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables
Trigger Functions
Types
Views
Subscriptions
Login/Group Roles
Tablespaces

3. Características de las bases de datos objeto-relacionales

Entre las características más importantes de las bases de datos objeto-relacionales (ORDBMS) se encuentra:

- **Soporte para tipos de datos complejos**: Permiten la definición de tipos de datos complejos, como objetos, colecciones, matrices, imágenes y otros tipos de datos complejos.
- **Herencia de tablas**: Permiten la definición de relaciones de herencia entre tablas, lo que significa que una tabla puede heredar atributos y comportamientos de otra tabla.
- **Polimorfismo**: Permiten la definición de métodos y operaciones que pueden ser aplicados a diferentes tipos de datos, lo que se conoce como polimorfismo.
- **Persistencia de objetos**: Permiten la persistencia de objetos, lo que significa que los objetos pueden ser almacenados en la base de datos y recuperados posteriormente.
- **Soporte para lenguajes de programación**: Proporcionan soporte para lenguajes de programación, como Java y C++, lo que permite a los desarrolladores crear y manipular objetos complejos directamente en la base de datos.
- **Integración con bases de datos relacionales**: Proporcionan una integración completa entre los modelos relacionales y orientados a objetos, lo que significa que se pueden utilizar tanto tablas como objetos para almacenar y manipular datos.
- **Eficiencia en la manipulación de datos complejos**: Permiten la manipulación eficiente de datos complejos, lo que mejora la eficiencia y la escalabilidad en aplicaciones que manejan grandes cantidades de datos.
- **Funciones y procedimientos almacenados**: Proporcionan mecanismos para definir y ejecutar funciones y procedimientos almacenados, lo que puede mejorar la eficiencia en el acceso a los datos y permitir la automatización de tareas complejas.

Veamos un ejemplo práctico:

Imagina un sistema de gestión inmobiliaria avanzado implementado en PostgreSQL (un ORDBMS), donde aprovechas todas las características que hemos visto:

1. **Soporte para tipos de datos complejos**

   Definimos un tipo compuesto address y un arreglo de imágenes:

```
CREATE TYPE address AS (
  calle    TEXT,
  ciudad   TEXT,
  coordenadas POINT[]  -- array de puntos geográficos
);
CREATE TABLE propiedades (
  id SERIAL PRIMARY KEY,
  direccion address,
  fotos BYTEA[]        -- varias imágenes por propiedad
);
```

2. **Herencia de tablas**

   Creamos una tabla base propiedades y dos subclases que heredan de ella:

```
CREATE TABLE viviendas (
  num_habitaciones INT
) INHERITS (propiedades);
CREATE TABLE locales_comerciales (
  metros_cuadrados INT
) INHERITS (propiedades);
```

3. **Polimorfismo**

   Definimos una función calcula_valor(obj ANYELEMENT) que, según el tipo real del objeto, invoca la versión adecuada:

```
CREATE FUNCTION calcula_valor(viviendas) RETURNS NUMERIC AS $$
  SELECT num_habitaciones * 10000;
$$ LANGUAGE sql;
```

```
CREATE FUNCTION calcula_valor(locales_comerciales) RETURNS NUMERIC AS $$
  SELECT metros_cuadrados * 1500;
$$ LANGUAGE sql;
```

4. **Persistencia de objetos**

Al insertar un registro en viviendas, todo el objeto (incluyendo su dirección y fotos) queda almacenado tal cual:

```
INSERT INTO viviendas (direccion, fotos, num_habitaciones)
VALUES (ROW('Av. Principal','CiudadX', ARRAY[(1.0,2.0),(3.0,4.0)]),
      ARRAY[decode('...','hex')], 3);
```

5. **Soporte para lenguajes de programación**

Creamos un trigger en PL/Python que envía notificaciones cuando cambia el precio de una propiedad:

```
CREATE FUNCTION notificar_cambio_precio() RETURNS trigger
LANGUAGE plpython3u AS $$
  plpy.notice(f"Propiedad {TD['new']['id']} ha cambiado de precio");
  return TD['new']
$$;

CREATE TRIGGER trg_precio AFTER UPDATE OF precio ON propiedades
  FOR EACH ROW EXECUTE FUNCTION notificar_cambio_precio();
```

6. **Integración con bases de datos relacionales**

Mantenemos tablas relacionales clásicas (clientes, contratos) y las relacionamos con nuestros objetos:

```
CREATE TABLE clientes (
  id SERIAL PRIMARY KEY, nombre TEXT
);
CREATE TABLE contratos (
  id SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES clientes(id),
  propiedad_id INT REFERENCES propiedades(id),
```

```
  fecha_inicio DATE
);
```

7. **Eficiencia en la manipulación de datos complejos**

   Indexamos las coordenadas y el arreglo de fotos para búsquedas rápidas:

```
CREATE INDEX ON propiedades USING GIST ((direccion).coordenadas);
CREATE INDEX ON propiedades USING GIN (fotos);
```

8. **Funciones y procedimientos almacenados**

   Definimos un procedimiento que agrupa propiedades por ciudad y devuelve un resumen:

```
CREATE PROCEDURE resumen_por_ciudad()
LANGUAGE plpgsql AS $$
BEGIN
  PERFORM city, COUNT(*) AS total
  FROM propiedades, LATERAL (SELECT (direccion).ciudad AS city) AS d
  GROUP BY city;
END;
$$;

CALL resumen_por_ciudad();
```

4. Tipos de datos objeto

4.1 Tipos de datos nativos en PostgreSQL

Los tipos de datos nativos son aquellos que provee directamente el SGBD sin que, como usuarios, tengamos que crearlos o definirlos.

Entre los datos nativos de PostgreSQL, encontramos:

- **Tipos de datos geométricos**: Estos tipos de datos son utilizados para almacenar datos relacionados con la geometría, como puntos, líneas y polígonos. Los tipos de datos geométricos en PostgreSQL incluyen POINT, LINE, LSEG, BOX, PATH, POLYGON y CIRCLE.
- **Tipos de datos de red**: Estos tipos de datos son utilizados para almacenar datos relacionados con la red, como direcciones IP y direcciones MAC. Los tipos de datos de red en PostgreSQL incluyen INET y MACADDR.
- **Tipos de datos de texto**: Estos tipos de datos son utilizados para almacenar datos de texto, como documentos de texto, correos electrónicos y otros tipos de datos relacionados con el lenguaje natural. Los tipos de datos de texto en PostgreSQL incluyen TEXT, VARCHAR y CHAR.
- **Tipos de datos binarios**: Estos tipos de datos son utilizados para almacenar datos binarios, como imágenes, archivos de sonido y otros tipos de datos que no son de texto. Los tipos de datos binarios en PostgreSQL incluyen BYTEA y OID.
- **Tipos de datos de fecha y hora**: Estos tipos de datos son utilizados para almacenar datos relacionados con la fecha y la hora. Los tipos de datos de fecha y hora en PostgreSQL incluyen DATE, TIME, TIMESTAMP y INTERVAL.
- **Tipos de datos numéricos**: Estos tipos de datos son utilizados para almacenar datos numéricos, como números enteros y decimales. Los tipos de datos numéricos en PostgreSQL incluyen INTEGER, BIGINT, NUMERIC y DECIMAL.

4.2. Tipos de datos personalizados

En PostgreSQL, los usuarios pueden definir tipos de datos personalizados para adaptar el esquema a las necesidades de su aplicación. Esto se logra con la sentencia estándar SQL CREATE TYPE, que permite crear:

- **Tipos compuestos** (composite), que agrupan varios campos de distinto tipo.
- **Tipos enumerados** (ENUM), con un conjunto finito de valores posibles.
- **Tipos de rango** (RANGE), que representan intervalos sobre un subtipo existente.
- **Arreglos** (arrays), añadiendo [] a cualquier tipo.

Por ejemplo:

```
-- Tipo compuesto
CREATE TYPE persona AS (
  nombre TEXT,
  edad  INT
);
-- Tipo enumerado
CREATE TYPE estado_pago AS ENUM ('pendiente','pagado','cancelado');
-- Tipo de rango
CREATE TYPE periodo AS RANGE (SUBTYPE = DATE);
```

Algunos ejemplos de tipos de datos personalizados que se pueden definir en PostgreSQL son:

Algunos ejemplos de tipos de datos personalizados que se pueden definir en PostgreSQL son:

- Tipos compuestos (COMPOSITE): permiten agrupar varios campos heterogéneos en un único tipo.

  Ejemplo:

  CREATE TYPE direccion AS (calle text, ciudad text, cp integer);

- Arrays: arrays unidimensionales o multidimensionales de un mismo tipo base.

  Ejemplo:

CREATE TABLE persona (id serial, nombres text[]);

- Enumerados (ENUM): definen un conjunto finito de valores permitidos.

Ejemplo:

CREATE TYPE estado_pedido AS ENUM ('pendiente','enviado','entregado');

- Rangos (RANGE): representan un intervalo contiguo de un tipo base (numérico, fecha, etc.).

Ejemplo:

CREATE TYPE rango_fechas AS RANGE (SUBTYPE = timestamp);

La creación de tipos de datos personalizados en PostgreSQL permite la adaptación de la base de datos a las necesidades específicas de cada aplicación, lo que puede mejorar la eficiencia y la calidad de los datos almacenados.

5. Definición de tipos de objeto

En PostgreSQL, los tipos de objetos se utilizan para definir estructuras de datos complejas que pueden contener diferentes tipos de datos, incluyendo otros objetos.

La definición de un tipo de objeto en PostgreSQL implica la creación de una nueva estructura de datos que se puede utilizar en las tablas y vistas de la base de datos.

Para definir un tipo de objeto se utiliza el lenguaje de definición de datos (DDL)

La sintaxis básica para la definición de un tipo de objeto en PostgreSQL es la siguiente:

```
CREATE TYPE nombre_del_tipo_de_objeto AS (lista_de_campos);
```

Donde **nombre_del_tipo_de_objeto** es el nombre que se le dará al tipo de objeto y **lista_de_campos** es la lista de campos que conformarán el objeto.

Cada campo debe estar compuesto por un nombre y un tipo de dato. Los campos se separan entre ellos con comas.

Importante

La definición de tipos de objetos en PostgreSQL permite la creación de estructuras de datos complejas que pueden mejorar la calidad y la eficiencia de la base de datos.

Ejemplo de creación de un tipo de dato:

```
CREATE TYPE direccion AS (
   calle TEXT,
   numero INTEGER,
   ciudad TEXT,
   pais TEXT
);
```

En este ejemplo, se está creando un tipo de objeto llamado **direccion** que contiene cuatro campos: **calle, numero, ciudad y pais**, cada uno con su tipo de dato correspondiente.

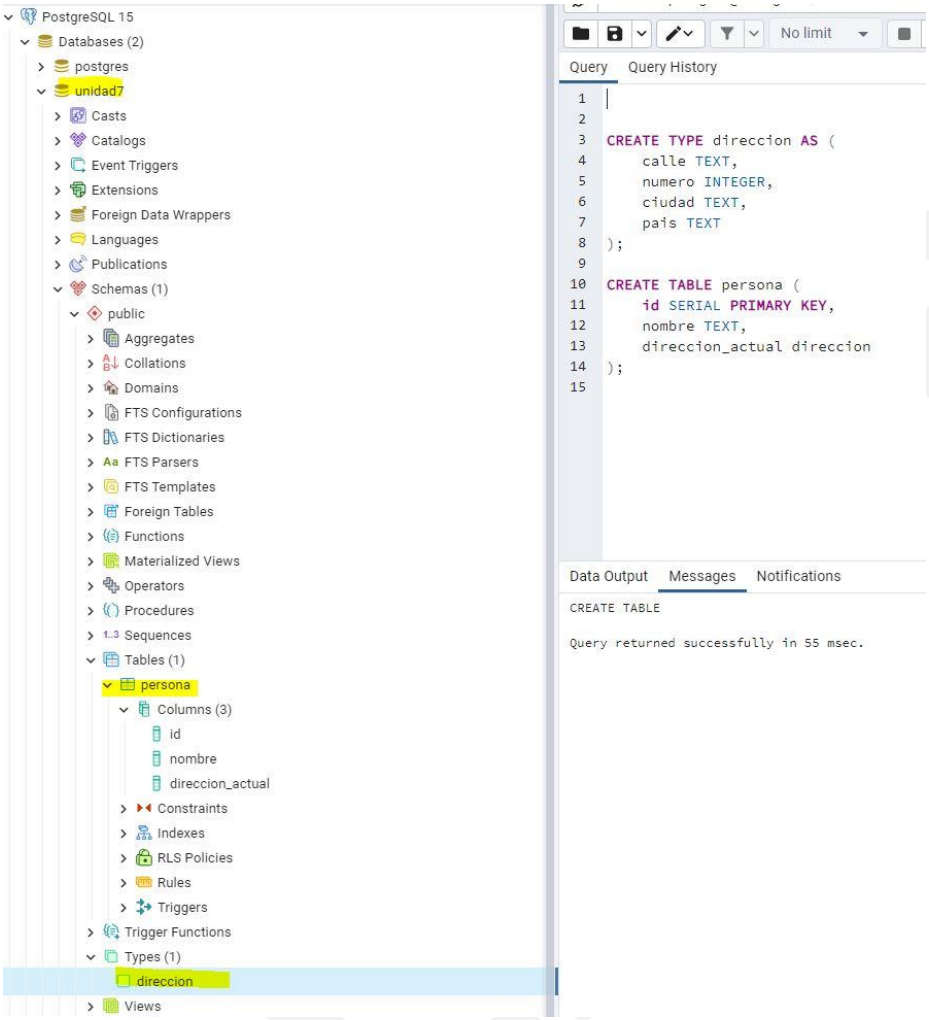
Una vez definido el tipo de objeto, se puede utilizar en las tablas y vistas de la base de datos, por ejemplo:

Ejemplo de uso de un tipo de dato en una tabla:

```
CREATE TABLE persona (
   id SERIAL PRIMARY KEY,
   nombre TEXT,
   direccion_actual direccion
);
```

En este ejemplo, se está creando una tabla llamada **persona** que contiene un campo llamado **direccion_actual** que utiliza el tipo de objeto **direccion** definido anteriormente.

En esta imagen podemos ver el resultado en pgAdmin4 de ejecutar los anteriores comandos:



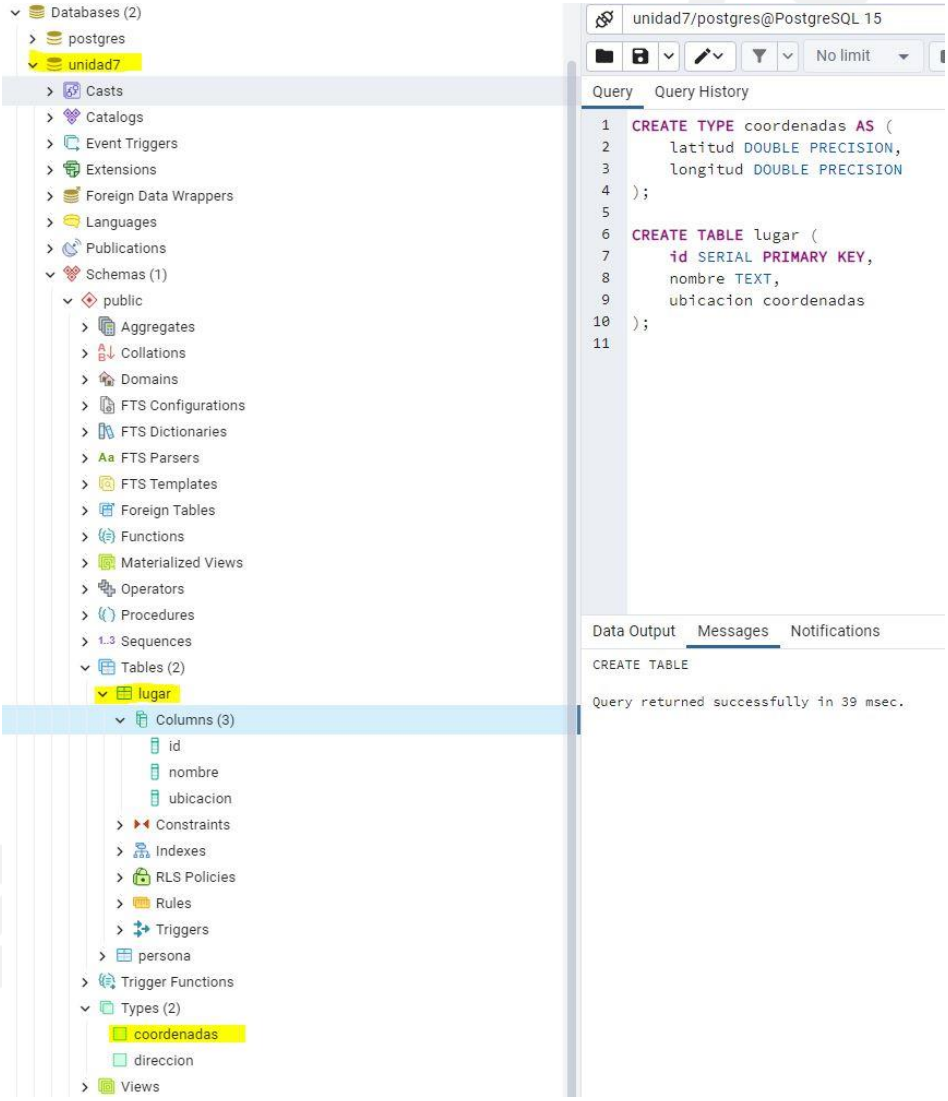
Ejemplo de creación de un tipo de dato y su uso en una tabla:

```
CREATE TYPE coordenadas AS (
   latitud DOUBLE PRECISION,
   longitud DOUBLE PRECISION
);
CREATE TABLE lugar (
   id SERIAL PRIMARY KEY,
   nombre TEXT,
   ubicacion coordenadas
);
```

En este ejemplo, se está creando un tipo de objeto llamado **coordenadas** que contiene dos campos: **latitud** y **longitud**, ambos de tipo DOUBLE PRECISION.

Luego, se crea una tabla llamada **lugar** que contiene un campo llamado **ubicación** que utiliza el tipo de objeto **coordenadas**. Este tipo de objeto puede ser útil en aplicaciones que requieren el almacenamiento de información geográfica.

Este es el resultado en pgAdmin4:



6. Herencia

6.1 Herencia de tablas

En PostgreSQL, la herencia permite definir una tabla "padre" con un conjunto de columnas y restricciones, y luego crear una o varias tablas "hijas" que heredan automáticamente esas mismas columnas y restricciones (como NOT NULL, DEFAULT o CHECK).

A las tablas hijas se les pueden añadir columnas y restricciones adicionales propias.

Hay que tener en cuenta que la clave primaria, los índices y las claves foráneas de la tabla padre **no** se copian automáticamente: si los necesitamos en la tabla hija, debemos declararlos de nuevo.

Importante

La **herencia en PostgreSQL** puede ser útil para simplificar el diseño de la base de datos y evitar la duplicación de campos y restricciones en varias tablas. Sin embargo, es importante tener en cuenta que la herencia no siempre es la mejor opción, y que puede ser necesario utilizar otras técnicas de modelado de datos para abordar situaciones más complejas.

La sintaxis básica para definir una tabla con herencia en PostgreSQL es la siguiente:

```
CREATE TABLE tabla_base (
   campo_1 tipo_de_dato_1,
   campo_2 tipo_de_dato_2,
   ...
);
CREATE TABLE tabla_hija (
   campo_3 tipo_de_dato_3,
   campo_4 tipo_de_dato_4,
   ...
) INHERITS (tabla_base);
```

En este ejemplo, se está creando una tabla base llamada **tabla_base** con campos campo_1 y campo_2.

Después, se crea una tabla hija llamada **tabla_hija** con campos campo_3 y campo_4, y se especifica que hereda los campos de la tabla base utilizando la cláusula **INHERITS**.

Las tablas hijas heredan los campos y restricciones de la tabla base, pero también pueden tener campos y restricciones adicionales.

Ejemplo de creación de herencia entre tablas:

```
CREATE TABLE empleados (
   id SERIAL PRIMARY KEY,
   nombre TEXT,
   salario NUMERIC(10, 2)
);
CREATE TABLE gerentes (
   oficina TEXT,
   departamento TEXT
) INHERITS (empleados);
```

En este ejemplo, se está creando una tabla base llamada **empleados** con campos id, nombre y salario.

A continuación, se crea una tabla hija llamada **gerentes** con campos adicionales oficina y departamento.

La tabla gerentes hereda los campos y restricciones de la tabla empleados, lo que significa que también tendrá los campos id, nombre y salario.

```
-- Tabla padre con clave primaria sobre id
CREATE TABLE empleados (
   id SERIAL PRIMARY KEY,
   nombre TEXT,
   salario NUMERIC(10,2)
```

```
);

-- Tabla hija que hereda columnas de empleados
-- Y vuelve a declarar la clave primaria sobre id
CREATE TABLE gerentes (
   oficina     TEXT,
   departamento TEXT,
   PRIMARY KEY (id)
) INHERITS (empleados);
```

Definimos primero la tabla **empleados** con las columnas id, nombre y salario, y establecemos id como clave primaria.

A continuación, creamos la tabla **gerentes**, que hereda automáticamente las columnas id, nombre y salario de **empleados** gracias a la cláusula INHERITS (empleados).

Como PostgreSQL **no** hereda restricciones de clave primaria ni índices automáticamente, volvemos a declarar PRIMARY KEY (id) en la tabla hija para que gerentes.id siga siendo única.

Una vez definida de este modo, podremos insertar registros en **gerentes** que comparten la estructura de **empleados** más sus propias columnas (oficina, departamento), y id se mantendrá como identificador único en ambas tablas.

6.2. Restricciones a la herencia en PostgreSQL

En PostgreSQL, existen algunas restricciones a tener en cuenta al utilizar la herencia en tablas:

La tabla hija puede heredar de una o varias tablas padre listándolas en la cláusula INHERITS. Por ejemplo:

```
CREATE TABLE empleado_completo (
  bonus NUMERIC(10,2)
) INHERITS (empleados, gerentes);
```

En este caso, empleado_completo hereda todas las columnas de empleados y de gerentes. Debemos tener en cuenta que índices y restricciones (como claves foráneas o únicas) **no** se copian automáticamente y deben definirse de nuevo si las necesitamos.

En PostgreSQL, existen algunas restricciones a tener en cuenta al utilizar la herencia en tablas:

- **La tabla hija puede heredar de una o varias tablas padre listándolas en la cláusula INHERITS**. Por ejemplo:

  ```
  CREATE TABLE empleado_completo (
    bonus NUMERIC(10,2)
  ) INHERITS (empleados, gerentes);
  ```

  En este caso, empleado_completo hereda todas las columnas de empleados y de gerentes. Debemos tener en cuenta que índices y restricciones (como claves foráneas o únicas) **no** se copian automáticamente y deben definirse de nuevo si las necesitamos.

- **La tabla hija no puede tener columnas con los mismos nombres que las columnas de la tabla padre**: esto se debe a que la herencia se basa en la suposición de que la tabla hija tiene columnas adicionales a las de la tabla padre. Si una columna de la tabla hija tiene el mismo nombre que una columna de la tabla padre, esto podría causar confusiones al realizar consultas.
- **Las restricciones de clave foránea solo se aplican a la tabla hija**: si una tabla padre tiene una restricción de clave foránea, esta restricción no se aplicará automáticamente a la tabla hija. En su lugar, se debe agregar manualmente una restricción de clave foránea a la tabla hija que haga referencia a la tabla padre.
- **Las restricciones de unicidad solo se aplican a la tabla hija**: de manera similar a la restricción de clave foránea, si una tabla padre tiene una restricción de unicidad, esta restricción no se aplicará automáticamente a la tabla hija. En su lugar, se debe agregar manualmente una restricción de unicidad a la tabla hija para evitar que se ingresen valores duplicados.

Es importante tener en cuenta estas restricciones al utilizar la herencia en PostgreSQL para evitar problemas de integridad de datos y consultas incorrectas.

6.3. Técnicas de herencia múltiple

A diferencia de muchos otros sistemas, PostgreSQL **sí** permite que una tabla herede directamente de varias tablas "padre" de forma nativa. Basta con listar todas las tablas de las que se quiere heredar en la cláusula INHERITS. Por ejemplo:

```
CREATE TABLE padre1 (
  id SERIAL PRIMARY KEY,
  campo1 TEXT
);
CREATE TABLE padre2 (
  id SERIAL PRIMARY KEY,
  campo2 TEXT
);
CREATE TABLE hija (
  campo_hija TEXT
) INHERITS (padre1, padre2);
```

En este caso, hija obtiene todos los campos (y sus restricciones) de padre1 y de padre2, además de su propia columna campo_hija.

Consideraciones:

- Las restricciones de clave primaria no se fusionan: si ambas tablas padre definen id como PK, la tabla hija heredará ambas columnas id (lo que suele requerir renombrarlas o redefinirlas).
- Las restricciones de unicidad o de clave foránea **no** se heredan automáticamente; deben crearse explícitamente en la tabla hija.
- Las consultas sobre la tabla padre, por defecto, no incluyen las filas de las hijas. Para ver todas en árbol, se usa SELECT * FROM padre1*; (nota: el asterisco tras el nombre).

Con esto, no es necesario recurrir a tablas intermedias ni a vistas para simular herencia múltiple en PostgreSQL.

Ejemplo de herencia múltiple "nativa" en PostgreSQL

Supongamos que en nuestra aplicación queremos modelar dos entidades base:

- persona con datos biográficos
- cuenta con datos de acceso

Y luego una entidad combinada empleado que deba reunir ambos conjuntos de campos más sus propios atributos.

```
-- Tabla padre 1: persona
CREATE TABLE persona (
  persona_id    SERIAL PRIMARY KEY,
  nombre        TEXT NOT NULL,
  fecha_nac     DATE
);

-- Tabla padre 2: cuenta
CREATE TABLE cuenta (
  cuenta_id     SERIAL PRIMARY KEY,
  usuario       VARCHAR(50) UNIQUE NOT NULL,
  correo        VARCHAR(100) NOT NULL
);

-- Tabla hija: empleado hereda de persona y de cuenta
CREATE TABLE empleado (
  empleado_id         SERIAL PRIMARY KEY,
  fecha_contratacion  DATE     NOT NULL,
  salario             NUMERIC(10,2)
) INHERITS (persona, cuenta);
```

¿Qué sucede aquí?

1. empleado obtiene automáticamente todos los campos de persona (persona_id, nombre, fecha_nac) y de cuenta (cuenta_id, usuario, correo).
2. Además define sus propias columnas: empleado_id, fecha_contratacion y salario.
3. Las claves primarias de las tablas padre no se fusionan; empleado mantiene sus propias PK y hereda columnas persona_id y cuenta_id (normalmente habría que renombrarlas o redefinirlas si queremos evitar duplicidad).

Cómo usarla

```
INSERT INTO empleado (nombre, fecha_nac, usuario, correo, fecha_contratacion, salario)
VALUES
  ('Ana López', '1985-07-12', 'alopez', 'ana@example.com', '2024-01-15', 2500.00);


SELECT * FROM empleado;
-- Muestra columnas persona.*, cuenta.* y empleado.*
```

7. Identificadores; referencias. Navegabilidad

En el contexto de bases de datos, los **identificadores** y las **referencias** son importantes para la navegabilidad de los datos.

Un identificador es una etiqueta única que se asigna a un objeto en una base de datos, mientras que una referencia es una conexión lógica entre dos objetos que indica la relación entre ellos.

Un identificador se utiliza para identificar de forma única un objeto en la base de datos, como una tabla, una columna o una clave primaria.

Por ejemplo, en una tabla de empleados, cada empleado tendría un identificador único, como un número de identificación de empleado.

Las referencias se utilizan para indicar la relación entre dos objetos en una base de datos.

Por ejemplo, si tenemos dos tablas, una de empleados y otra de departamentos, podemos crear una referencia entre ellas para indicar qué empleado pertenece a qué departamento. Esto se hace mediante una clave foránea en la tabla de empleados que hace referencia a la clave primaria en la tabla de departamentos.

La **navegabilidad en una base de datos** se refiere a la capacidad de seguir las referencias entre los objetos en la base de datos.

Por ejemplo, si tienes una tabla de departamentos y una tabla de empleados, y una referencia entre ellas que indica qué empleado pertenece a qué departamento, podemos navegar desde un departamento a los empleados que pertenecen a ese departamento.

La navegabilidad también se puede utilizar para seguir las referencias entre tablas relacionadas en una base de datos.

Por ejemplo, si tenemos una tabla de clientes y una tabla de pedidos, podemos seguir la referencia entre ellas para ver los pedidos realizados por un cliente en particular.

Importante

Los identificadores y las referencias son importantes para la navegabilidad en una base de datos.

Los identificadores se utilizan para identificar de forma única los objetos en la base de datos, mientras que las referencias se utilizan para indicar la relación entre los objetos y permitir la navegación entre ellos.

Ejemplo de identificadores, referencias, y navegabilidad:

Supongamos que tenemos dos tablas en PostgreSQL: una tabla clientes y una tabla pedidos, y que cada cliente puede tener varios pedidos, pero cada pedido solo puede pertenecer a un cliente.

Podemos usar identificadores y referencias para establecer esta relación y permitir la navegabilidad entre las dos tablas.

Primero, creamos la tabla clientes:

```
CREATE TABLE clientes (
   id_cliente SERIAL PRIMARY KEY,
   nombre VARCHAR(50),
   email VARCHAR(50),
   direccion VARCHAR(100)
);
```

La columna id_cliente es la clave primaria de la tabla clientes y se usa como identificador único para cada cliente.

A continuación, creamos la tabla pedidos:

```
CREATE TABLE pedidos (
   id_pedido SERIAL PRIMARY KEY,
   id_cliente INTEGER REFERENCES clientes(id_cliente),
   fecha_pedido DATE,
   total DECIMAL(10, 2)
);
```

La columna id_pedido es la clave primaria de la tabla pedidos y se utiliza como identificador único para cada pedido.

La columna id_cliente es una clave foránea que hace referencia a la columna id_cliente de la tabla clientes, estableciendo así la relación entre las dos tablas.

Ahora, insertamos algunos datos en las dos tablas para establecer una relación entre ellas:

```
INSERT INTO clientes (nombre, email, direccion) VALUES
   ('Juan Perez', 'juan.perez@example.com', 'Calle 123'),
   ('Maria Lopez', 'maria.lopez@example.com', 'Calle 456');

INSERT INTO pedidos (id_cliente, fecha_pedido, total) VALUES
   (1, '2023-04-30', 100.00),
   (2, '2023-04-29', 50.00),
   (1, '2023-04-28', 200.00);
```

En este ejemplo, se insertan dos clientes y tres pedidos.
El primer pedido pertenece al cliente con id_cliente 1, el segundo pedido pertenece al cliente con id_cliente 2 y el tercer pedido también pertenece al cliente con id_cliente 1.

Ahora podremos usar la navegabilidad para acceder a los datos de ambas tablas.
Por ejemplo, para obtener todos los pedidos de un cliente en particular, podemos usar la referencia entre las dos tablas de la siguiente manera:

```
SELECT * FROM pedidos WHERE id_cliente = 1;
```

Esto devolverá los dos pedidos que pertenecen al cliente con id_cliente 1.
En este ejemplo hemos visto cómo podemos utilizar identificadores y referencias para establecer relaciones entre tablas en PostgreSQL y permitir la navegabilidad entre ellas.

8. Tipos de datos colección

Definición

Los tipos de datos de colección son tipos de datos en los que se pueden almacenar múltiples valores en una sola variable o columna de una tabla.

Estos tipos de datos son comunes en muchos lenguajes de programación y sistemas de bases de datos.

En PostgreSQL, los tipos de datos de colección más comunes son los siguientes:

- **Array**

Colección ordenada de elementos del mismo tipo. Puede ser multidimensional, por ejemplo:

INTEGER[] — array unidimensional de enteros

INTEGER[][] — array bidimensional de enteros

- **Rango**

Intervalo de valores continuos. PostgreSQL proporciona varios tipos de rango nativos, por ejemplo:

daterange — rango de fechas

int4range — rango de enteros de 4 bytes

- **JSON / JSONB**

Permiten almacenar documentos JSON completos, con listas y objetos arbitrarios:

```
CREATE TABLE config (
  id SERIAL PRIMARY KEY,
  opciones JSONB
);
```

- **ENUM**

Conjunto finito de valores posibles, definido previamente como tipo enumerado:

```
CREATE TYPE dia_semana AS ENUM (
  'Lunes','Martes','Miércoles','Jueves','Viernes','Sábado','Domingo'
);
```

Importante
Elegir el tipo de colección adecuado según el caso de uso:

- **ARRAY** para listas simples y multidimensionales de un solo tipo.
- **RANGE** para representar intervalos continuos.
- **JSON/JSONB** para estructuras flexibles y heterogéneas.
- **ENUM** para listas cortas de valores fijos y conocidos de antemano.

Importante

Los tipos de datos de colección en PostgreSQL permiten almacenar múltiples valores en una sola variable o columna de una tabla, lo que puede simplificar el diseño de la base de datos y mejorar la eficiencia del acceso a los datos.

9. Declaración e inicialización de objetos

En PostgreSQL, podemos **declarar e inicializar diferentes objetos de base de datos**, como tablas, vistas, funciones, índices, etc.
La sintaxis para la declaración e inicialización varía según el tipo de objeto. A continuación, se presentan algunos ejemplos comunes:

Tablas:
Para crear una tabla en PostgreSQL, podemos usar la siguiente sintaxis:

```
CREATE TABLE nombre_tabla (
  columna1 tipo_dato_restricciones,
  columna2 tipo_dato_restricciones,
  ...
);
```

Por ejemplo, para crear una tabla usuarios con columnas id, nombre y email, puedes usar:

```
CREATE TABLE usuarios (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL,
  email TEXT UNIQUE
);
```

Vistas:
Para crear una vista en PostgreSQL, podemos usar la siguiente sintaxis:

```
CREATE VIEW nombre_vista AS
SELECT columna1, columna2, ...
FROM nombre_tabla
WHERE condicion;
```

Por ejemplo, para crear una vista vista_usuarios que muestre solo los nombres y correos electrónicos de la tabla usuarios, podríamos usar:

```
CREATE VIEW vista_usuarios AS
SELECT nombre, email
FROM usuarios;
```

Funciones:
Para crear una función en PostgreSQL, podemos usar la siguiente sintaxis:

```
CREATE FUNCTION nombre_funcion(parametro1 tipo_dato, parametro2 tipo_dato, ...)
RETURNS tipo_dato
AS $$
```

```
-- cuerpo de la función
BEGIN
  -- código
END;
$$ LANGUAGE plpgsql;
```

Por ejemplo, para crear una función mayúscula que convierta una cadena de texto en mayúsculas, podríamos usar:

```
CREATE FUNCTION mayusculas(texto TEXT)
RETURNS TEXT
AS $$
BEGIN
  RETURN UPPER(texto);
END;
$$ LANGUAGE plpgsql;
```

Índices:

Para crear un índice en PostgreSQL, podemos usar la siguiente sintaxis:

```
CREATE INDEX nombre_indice
ON nombre_tabla (columna1, columna2, ...);
```

Por ejemplo, para crear un índice indice_usuarios_email en la columna email de la tabla usuarios, podríamos usar:

```
CREATE INDEX indice_usuarios_email
ON usuarios (email);
```

Importante

Debemos tener en cuenta que la sintaxis y las opciones pueden variar según el tipo de objeto y las necesidades específicas de tu aplicación.

Algunos de estos objetos, además de definirse, pueden inicializarse, como en el caso de las tablas mediante la inserción de filas, que veremos más adelante.

10. Inserción de objetos

En PostgreSQL, las tablas pueden heredar de una tabla padre utilizando el mecanismo de herencia. Esto permite a las tablas hijas compartir las columnas de la tabla padre y también añadir sus propias columnas adicionales.

Importante

Cuando se inserta un registro en una tabla hija, los valores correspondientes se insertan automáticamente en la tabla padre.

En general, el proceso de inserción de objetos en tablas con herencia en PostgreSQL sigue los mismos pasos que la inserción en tablas regulares. Se utiliza la sintaxis INSERT INTO <nombre de tabla> (<columnas>) VALUES (<valores>) para insertar un nuevo registro en la tabla correspondiente.

La única diferencia es que al insertar un registro en una tabla hija, se insertan automáticamente valores en la tabla padre correspondiente.

Importante

Al utilizar la herencia en PostgreSQL, se deben establecer las relaciones de clave foránea adecuadas para que los registros se inserten correctamente en todas las tablas relacionadas. En PostgreSQL, **no** hay cascada automática de borrado al usar herencia; es necesario declarar ON DELETE CASCADE en las claves foráneas de las hijas (o gestionar manualmente)

Ejemplo:

Supongamos que tenemos una tabla personas como tabla padre y dos tablas hijas estudiantes y profesores.
La tabla personas tiene las columnas id, nombre y apellido, mientras que la tabla estudiante tiene una columna adicional llamada curso y la tabla profesores tiene una columna adicional llamada especialidad.
La declaración de las tablas sería la siguiente:

```
CREATE TABLE personas (
   id SERIAL PRIMARY KEY,
   nombre TEXT,
   apellido TEXT
);
```

```
CREATE TABLE estudiantes (
   curso INTEGER
) INHERITS (personas);

CREATE TABLE profesores (
   especialidad TEXT
) INHERITS (personas);
```

Para insertar un nuevo registro en la tabla estudiantes, se puede utilizar la siguiente sintaxis:

```
INSERT INTO estudiantes (nombre, apellido, curso) VALUES ('Pedro', 'Fernández', 2);
```

En este caso, se inserta un nuevo registro en la tabla personas con los valores 'Pedro' y 'Fernández' en las columnas nombre y apellido, respectivamente, y se inserta un nuevo registro en la tabla estudiantes con el valor 2 en la columna curso.

Si se desea insertar un nuevo registro en la tabla profesores, se puede utilizar la siguiente sintaxis:

```
INSERT INTO profesores (nombre, apellido, especialidad) VALUES ('Marta', 'Ramírez', 'Latín');
```

En este caso, se inserta un nuevo registro en la tabla personas con los valores 'Marta' y 'Ramírez' en las columnas nombre y apellido, respectivamente, y se inserta un nuevo registro en la tabla profesores con el valor 'Latín' en la columna especialidad.

11. Uso de la sentencia SELECT

En PostgreSQL, la sentencia SELECT se puede utilizar para recuperar datos de una tabla que tiene herencia con otras tablas.

Importante

Cuando se ejecuta una consulta SELECT en una tabla que tiene herencia, se incluyen los registros de la tabla hija y los registros de todas las tablas padre relacionadas. Esto permite una vista unificada de todos los registros de la tabla y sus tablas relacionadas.

Ejemplo:

Si a continuación, ejecutamos las siguientes consultas, veremos el resultado de las inserciones del ejemplo del apartado en el que vimos las inserciones de registros.

Si ejecutamos la siguiente consulta:
SELECT * FROM personas;

Obtendremos todos los registros de la tabla padre personas, así como todos los registros de las tablas hijas estudiantes y profesores.

También podemos ejecutar consultas en una tabla hija específica utilizando la sintaxis de SELECT estándar.

Por ejemplo, si queremos recuperar todos los registros de la tabla estudiantes, podemos ejecutar la siguiente consulta:
SELECT * FROM estudiantes;

Esta consulta solo incluirá los registros de la tabla estudiantes, no los de la tabla profesores o personas.

Y lo mismo ocurrirá si recuperamos los registros de la tabla profesores:
SELECT * FROM profesores;

```
SELECT * from personas;
```

Output Messages Notifications

id [PK] integer	nombre text	apellido text
1	Pedro	Fernández
2	Marta	Ramírez

```
SELECT * from estudiantes;
```

Output Messages Notifications

id integer	nombre text	apellido text	curso integer
1	Pedro	Fernández	2

```
SELECT * from profesores;
```

Output Messages Notifications

id integer	nombre text	apellido text	especialidad text
2	Marta	Ramírez	Latín

Además, se pueden utilizar otras cláusulas de la sentencia SELECT como JOIN, WHERE, ORDER BY, GROUP BY y HAVING para realizar consultas más complejas que involucren tablas con herencia.

Importante

La **sentencia SELECT en PostgreSQL** funciona de manera similar en las tablas con herencia que, en las tablas normales, pero incluye registros de todas las tablas relacionadas (padres e hijos) de forma predeterminada.

12. Modificación y borrado de objetos

En PostgreSQL, la **modificación y eliminación de objetos** se realizan utilizando las sentencias UPDATE, DELETE y DROP. A continuación, se explican brevemente estas sentencias:

UPDATE: se utiliza para modificar uno o varios registros en una tabla. La sintaxis básica es la siguiente:
UPDATE nombre_tabla SET columna1 = valor1, columna2 = valor2 WHERE condicion;
Donde nombre_tabla es el nombre de la tabla a modificar, columna1 y columna2 son los nombres de las columnas a modificar, valor1 y valor2 son los nuevos valores para esas columnas, y condición es una expresión que especifica qué registros deben ser modificados.

DELETE: se utiliza para eliminar uno o varios registros en una tabla.
La sintaxis básica es la siguiente:
DELETE FROM nombre_tabla WHERE condicion;
Donde nombre_tabla es el nombre de la tabla a eliminar registros, y condición es una expresión que especifica qué registros deben ser eliminados.

DROP: se utiliza para eliminar un objeto de la base de datos (tabla, función, vista, etc.).
La sintaxis básica es la siguiente:
DROP objeto nombre_objeto;
Donde objeto es el tipo de objeto que se desea eliminar (TABLE, FUNCTION, VIEW, etc.), y nombre_objeto es el nombre del objeto a eliminar.

Importante

Si una tabla tiene herencia con otras tablas, la modificación o eliminación de un registro puede afectar a todas las tablas relacionadas. Si modificamos una o varias columnas de una tabla hija, estas modificaciones también afectarán a las columnas de todas las tablas padre.Si eliminamos un registro de una tabla hija, también se eliminarán los registros correspondientes de todas las tablas padre. Por lo tanto, es necesario tener cuidado al realizar modificaciones o eliminaciones en tablas con herencia, y asegurarse de entender completamente cómo funciona la herencia en PostgreSQL.

Ejemplo UPDATE:

Si partimos de la situación actual de las tablas personas, estudiantes, y profesores, vemos que su contenido es el siguiente:

```
SELECT * from personas;
```

Output Messages Notifications

id [PK] integer	nombre text	apellido text
1	Pedro	Fernández
2	Marta	Ramírez

```
SELECT * from estudiantes;
```

Output Messages Notifications

id integer	nombre text	apellido text	curso integer
1	Pedro	Fernández	2

```
SELECT * from profesores;
```

Output Messages Notifications

id integer	nombre text	apellido text	especialidad text
2	Marta	Ramírez	Latín

Si aplicamos las siguientes modificaciones, veremos qué cambios se producen en dicho contenido:

```
UPDATE estudiantes
SET nombre='Manuel', curso=6
WHERE id=1;

UPDATE profesores
SET apellido='Álvarez', especialidad='Filosofía'
WHERE id=2;
```

Ahora volvemos a mostrar el contenido:

SELECT * FROM personas;
SELECT * FROM estudiantes;
SELECT * FROM profesores;

```
SELECT * FROM Personas;
```

Output Messages Notifications

id [PK] integer	nombre text	apellido text
1	Manuel	Fernández
2	Marta	Álvarez

```
SELECT * FROM estudiantes;
```

Output Messages Notifications

id integer	nombre text	apellido text	curso integer
1	Manuel	Fernández	6

```
SELECT * FROM profesores;
```

Output Messages Notifications

id integer	nombre text	apellido text	especialidad text
2	Marta	Álvarez	Filosofía

13. Resumen

PostgreSQL es un sistema de gestión de bases de datos objeto-relacional que soporta tipos de datos objeto, que permiten definir objetos complejos con atributos y métodos. Además, cuenta con la capacidad de definir nuevos tipos de objetos personalizados, a través de la definición de tipos de objeto en el lenguaje PL/SQL.

La herencia es otra característica importante de PostgreSQL, que permite definir una tabla en términos de otra, heredando sus columnas y características, lo que permite una mayor flexibilidad y reutilización de código.

Los identificadores y referencias permiten definir relaciones entre objetos, lo que permite la navegación entre ellos y la definición de restricciones y validaciones.

Los tipos de datos colección, como los conjuntos y los arrays, permiten almacenar múltiples valores en una sola columna.

La declaración e inicialización de objetos en PostgreSQL se realiza a través de sentencias CREATE y ALTER, que permiten definir y modificar objetos como tablas, funciones y vistas.

La inserción de objetos se realiza a través de sentencias INSERT, que permiten añadir registros a una tabla. La sentencia SELECT se utiliza para consultar los datos almacenados en las tablas, y es compatible con las tablas que utilizan la herencia.

La modificación y eliminación de objetos se realiza a través de sentencias UPDATE y DELETE, que permiten modificar y eliminar registros de las tablas, así como también eliminar objetos como tablas y vistas.

Tech Hub <FP> euroformac

Glosario

- **Base de datos objeto-relacional**: tipo de base de datos que combina características de las bases de datos relacionales y orientadas a objetos.

- **Herencia**: característica que permite definir una tabla en términos de otra, heredando sus columnas y características.

- **Tipo de datos objeto**: tipo de dato que permite definir objetos complejos con atributos y métodos.

- **PL/SQL**: lenguaje de programación utilizado en PostgreSQL para definir tipos de objetos personalizados.

- **Identificadores**: atributos que permiten identificar a los objetos en la base de datos.

- **Referencias**: relaciones entre objetos que permiten la navegación entre ellos y la definición de restricciones y validaciones.

- **Conjuntos**: tipo de datos colección que permite almacenar múltiples valores sin repetición.

- **Arrays**: tipo de datos colección que permite almacenar múltiples valores con repetición.

- **Sentencia CREATE**: sentencia utilizada para crear objetos en PostgreSQL, como tablas, funciones y vistas.

- **Sentencia INSERT**: sentencia utilizada para insertar registros en una tabla.

- **Sentencia SELECT**: sentencia utilizada para consultar los datos almacenados en una tabla.

- **Sentencia UPDATE**: sentencia utilizada para modificar registros en una tabla.

- **Sentencia DELETE**: sentencia utilizada para eliminar registros en una tabla.

Bibliografía

Vázquez Ortíz, Y. & Sotolongo León, A. R. (2020). Fundamentos para el trabajo con PostgreSQL.

Ramos Serrano, C. (2021). Aprende PostgreSQL de principiante a avanzado 2021: De bases de datos orientada a la relación de los objetos.

Orbegozo Arana, B. & Casaleiz Fuentes, A. (2015). Curso práctico completo de PostgreSQL: la base de datos más potente

Tech Hub <FP> euroformac